ndhmd4tex.py Markdown Dialect — User Guide

Nova de Hi

2026년 11월 9일

This document is a reference guide to the markdown → LaTeX conversion syntax recognized by `ndhmd4tex.py`. It diverges from standard Pandoc markdown in several places, so be sure to check the sections on **smart quotes**, **list continuation**, **description lists** (`::`), and **custom environments** (`:::`) before relying on them.

> **Note on Korean-specific features**: this tool was originally built for editing Korean-language books, so a few sections (quote handling, the Josa particle macros) inherently involve Hangul (Korean script) text in their examples, since that's what those features operate on. Everything else is plain English.

---

1. How to run it

```
python3 ndhmd4tex.py input.md                  # produces input.tex
python3 ndhmd4tex.py input.md -o output.tex     # explicit output path
python3 ndhmd4tex.py input.md --config mapping.yaml   # override the markup<->command mappi
python3 ndhmd4tex.py input.md --setup base.md   # use this file's header as the base (see 1
python3 ndhmd4tex.py input.md --setup a.md,b.md   # merge several files in order (see 21.3)
python3 ndhmd4tex.py --init-setup               # write out the default header and exit
python3 ndhmd4tex.py input.txt --encoding cp949  # force the input encoding
python3 ndhmd4tex.py input.md --lint       # check for common markup mistakes (see 1.3)
python3 ndhmd4tex.py input.md --fetch-images  # download images given by URL (see 15.1)
python3 ndhmd4tex.py --search-resource          # the script/executable's own location
python3 ndhmd4tex.py --search-resource base.nmd # find it across current dir + resource dir
python3 ndhmd4tex.py input.md --pdf     # compile with pdflatex and open the viewer
python3 ndhmd4tex.py input.md --pdfxe   # compile with xelatex
python3 ndhmd4tex.py input.md --pdflua  # compile with lualatex
python3 ndhmd4tex.py input.md --pdfxe --brief  # hide the compiler's own output, keep just
python3 ndhmd4tex.py --version          # print the tool's version
python3 ndhmd4tex.py input.md --dump-meta   # print the merged front matter and exit
python3 ndhmd4tex.py input.md --pandoc      # hand the body to pandoc (see 1.4)
python3 ndhmd4tex.py input.md --pandoc="--number-sections"   # with pandoc arguments
```

If `--config` is omitted, the script looks for a `mapping:` key in the front matter (relative paths are resolved against the input file) and uses that automatically. The `--pdf`-family flags locate `ksruncltex.lua` via `kpsewhich` and run it through `texlua`, appending the front matter's `compileoption:` string verbatim as extra shell arguments. Once compilation finishes, the resulting PDF opens in the platform's default viewer (`open` on macOS, `xdg-open` on Linux, the registered handler on Windows). To use a different viewer, set the `NDHMARKDOWN_PDF_VIEWER` environment variable to the executable and any arguments (e.g. `okular --unique`). If the named program can't be found or fails to run, a warning is printed and the platform default is used instead.

Console output comes in two flavors. `--verbose` (the default, no need to type it) is exactly what's described above — with `--pdf`-family flags or `--latexmk`, the compiler's own log (the command line, xelatex/latexmk's progress) streams straight to the screen. **`--brief` hides that compiler log and shows only the essentials** — "컴파일합니다: (the actual command line)"/"PDF를 엽니다: …" and the like — except when compilation fails, in which case the hidden log is shown after all (being quiet and leaving no clue on failure are two different problems).

`--nopreview` compiles as usual but skips opening the PDF viewer afterwards (use it with `--pdf`/`--pdfxe`/`--pdflu`
It is handy when a viewer is already open, or in scripts/CI that only need the PDF. It combines with `--brief`.

Every message this tool prints itself (errors, warnings, notices — under either `--verbose` or `--brief`) is automatically prefixed with `[ndhmd]` , so console output can be filtered with something like `grep '\[ndhmd\]'` to see only this tool's own progress. The compiler's own log (streaming under `--verbose`, or shown only on failure under `--brief`) never gets this prefix — "what this tool said" and "what the compiler said" stay distinguishable. `--help`/`--version` output has no prefix either, to keep the standard help/version format intact.

Input files are decoded as **UTF-8 first, falling back to `cp949` and then `euc-kr`** — `.txt` manuscripts written on a Korean Windows system are commonly cp949. A warning is printed whenever the fallback is used, so if the text comes out garbled, name the encoding with `--encoding` or re-save the file as UTF-8. The same rule applies to `{{included}}` files and `setup:` files.

A mistyped filename, a directory, or a non-text file such as a PDF produces a plain explanation and exit status 1 rather than a traceback.

Line endings are normalized to LF on read. Left as CRLF — the default for Windows editors — a trailing `\r` makes the line-oriented rules miss silently: a heading's `{label=...}` survives as literal text in the title, or a closing code fence is not recognized and swallows the rest of the document.

`--dump-meta` performs no conversion: it prints the final front matter — with the whole `setup:` chain already merged in — as YAML and exits. Since a setting can come from any of several places (CLI / the current file / a setup file / built-in defaults), use this to answer "where on earth did this font setting come from?" (see section 21).

`--dump-mapping` prints the fully merged mapping rules from `mapping:`/`mappingrule:`/`--config`, as YAML, and exits. What `--dump-meta` shows for `mapping:` is only the front matter value itself — a **path** (or list of paths) to YAML files — not what you get once those files are actually read and merged. With several `mapping.yaml` files listed, some keys concatenate (like `extra_inline`) while others follow "whichever comes last wins," and `mappingrule:` and `--config` then layer on top in priority order — so to know what `**bold**` actually turns into right now, seeing the merged result directly beats comparing the files by eye.

```
python3 ndhmd4tex.py input.md --dump-mapping
```

Give both `--dump-meta` and `--dump-mapping` together and they print **in that order** (meta first, then mapping) to the console, and exit — either one alone, or both together, skips the actual conversion. Each one's output ends with a `====== end of meta =======` or `====== end of mapping =======` line, so when viewing both at once it's clear at a glance where one ends and the other begins.

1.1. Headerless manuscripts and the default header (`--setup`, `--default`)

This is what makes a manuscript with no front matter at all typeset properly. The base header is looked for in this order:

1. The file given as `--setup FILE` (comma-separate several, see 21.3). It **takes precedence** over the document's own `setup:` (the same principle by which `--config` beats the front matter's `mapping:`).
2. The document's own `setup:` (section 21).

3. The file given as `--default FILE`, or (if not given) `ndhmd-default.nmd` — first in the **current working directory**, then in the **directory holding the program**. This file is meant to ship alongside the script. A PyInstaller executable looks in slightly different places (see just below).
4. If none of those exist, it falls back to a bare `\documentclass{book}` as before.

Whatever is found is treated exactly like a `setup:` target, which means **the document's own front matter overrides it**. It is a default, not a mandate. `--dump-meta` shows the chain, so you can see where a setting came from.

```
$ python3 ndhmd4tex.py headerless.md --dump-meta
# setup 체인: headerless.md -> /home/nova/ndhmd-default.nmd
class: oblivoir
...
```

The body of `ndhmd-default.nmd` is never read — only its front matter — so notes below it are harmless. The extension is deliberately not `.md`, to keep it distinguishable from ordinary manuscripts in an editor.

`--default` (short form `-d`) temporarily swaps out slot 3 above for a different file. It's a different kind of thing from `--setup` — `--setup` is a top-priority order to use this header for this run no matter what, overriding even the document's own `setup:`. `--default` is much weaker: it only changes which file gets used as the fallback for documents that haven't specified any header of their own — a document that sets its own `setup:` still wins (the precedence between steps 2 and 3 above is unchanged).

```
python3 ndhmd4tex.py doc.md --default company-style    # looks for company-style.nmd
python3 ndhmd4tex.py doc.md -d company-style.nmd        # already has an extension, kept as-
```

- If there's no extension, `.nmd` is appended automatically. If there already is one, it's kept as given.
- It's searched for in exactly the same places `ndhmd-default.nmd` would be (working directory → next to the script/exe → shared resource folders). Give an absolute path and only that path is checked.
- This shows up in `--dump-meta` too — if the file named by `--default` wasn't found, that fact appears directly in the diagnostic message (rather than a bare "couldn't find ndhmd-default.nmd" that has nothing to do with what was actually asked for).

`--default NONE` — drop the document structure, keep the markup rules

`--default NONE` (exactly uppercase) is a far stronger, entirely different instruction from the above. It isn't just "put nothing in the default-header slot" — it means **don't build a document wrapper at all, just convert the markdown body to a LaTeX fragment.** No `\documentclass`, no `\begin{document}`/`\end{document}`.

A typical use case: a co-author already manages `main.tex` through pandoc, and you don't have write access to that file — you only need to hand over the **body content** in a form that fits their document. Set `--config`/`mapping:` to match their conventions (heading levels, custom syntax, etc.) and hand over just the fragment.

```
python3 ndhmd4tex.py doc.md --default NONE -o fragment.tex
```

```
\section{Title}

Body with \emph{emphasis} and \hl{highlighted}.
```

The result **doesn't compile on its own** — there's no document wrapping it. It's meant to be spliced into another document via `{{tex:fragment.tex}}`, or wrapped by hand with your own preamble.

Only the "document structure" is dropped — "how markup gets translated" is untouched. These are two different layers.

- **Dropped (automatically)**: `title`, `class`, `preamble`, `author`, `date`, `setup:`, `maketitle`, `tocloft`. These keys are only ever read by the code that assembles the document wrapper, so if that wrapper is never built, they naturally have nowhere to take effect. Front matter isn't skipped — it's still resolved normally (setup: chain included) — it's just that these particular keys end up with nothing to influence.
- **Still applied**: `mapping:`, `mappingrule:`, `toplevel:`, `pandocoption:`, `--config`. These govern how the body's markup gets translated, which is exactly what's needed to match the conventions of whatever document the fragment is headed for (heading levels, custom syntax added via `extra_inline`, etc.). Giving `--config` applies that mapping as usual.
- **`--pdf`/`--pdfxe`/`--pdflua`/`--latexmk` are all ignored** — there's no complete document to compile. A warning is printed and compilation is skipped entirely.
- **`--syncinfo` is meaningless as well** — with no compiled output, there's nothing for line numbers to correspond to. A warning is printed and no `.nmdsync` is written.
- **It does work together with `--pandoc`** — body conversion is handed to pandoc, but the result is still just the bare body, with no `\documentclass`/`\begin{document}`. This dialect's own markup extensions (like `extra_inline`) still don't apply here though — same existing limitation as `--pandoc` mode in general, since pandoc takes over the whole body conversion.
- Adding `--dump-meta` shows the fully resolved meta, with a note that document structure is dropped while markup-translation rules still apply.

Shipping as an executable

Under PyInstaller, `__file__` does not point next to the executable. Measured:

Build	`__file__`	`sys.executable`
`--onefile`	the temporary extraction dir (`Temp\_MEIxxxx`)	the exe itself
`--onedir`	`appdir\_internal`	`appdir\app.exe`

The `--onefile` temporary directory disappears when the run ends, so a user cannot even put a file there. When frozen, the search therefore uses the **directory holding the executable**:

1. the current working directory
2. the directory holding `ndhmd4tex.exe`
3. the data directory bundled inside the exe (`sys._MEIPASS`)

Thanks to (3) the **default header can ship inside the executable**. On Windows, bundle it like this — note the `;` separator for data files, where Linux and macOS use `:`.

```
pyinstaller --add-data="ndhmd-default.nmd;." --onefile ndhmd4tex.py
```

Editing the `.spec` file directly, that is the one line:

```
datas=[('ndhmd-default.nmd', '.')],
```

Now the default applies with nothing for the user to install, a file placed next to the exe overrides it, and a file in the project folder overrides that.

`--init-setup` — extracting the bundled header

With `--onefile` the default header lives inside the exe, where a user can neither read nor edit it. `--init-setup` writes a copy out.

```
D:\work> ndhmd4tex.exe --init-setup
D:\work\ndhmd-default.nmd 를 만들었습니다.  (원본: C:\...\Temp\_MEI123456\ndhmd-default.nmd)
```

Since the working directory comes first in the search order, that copy immediately shadows the bundled one. An existing file of the same name is never overwritten — the command fails instead. No input file is required.

When no default header is found, `--dump-meta` reports where it looked.

```
$ ndhmd4tex.exe doc.md --dump-meta
# ndhmd-default.nmd 을 찾지 못했습니다. 찾아본 곳:
#   D:\work\ndhmd-default.nmd
#   C:\Program Files\ndhmd\ndhmd-default.nmd
#   C:\Users\...\Temp\_MEI123456\ndhmd-default.nmd
```

Two things to watch.

- This does not apply only to headerless documents. It applies to **every** document without a `setup:` key. Dropping an `ndhmd-default.nmd` into a working directory therefore affects every manuscript in it — exactly what you want when a whole book lives in one folder, but capable of quietly changing other manuscripts too.
- Overriding `class` while leaving `classoptions` alone will not work. If the default header passes a `chapter` option and the document only sets `class: article`, you get `\documentclass[...,chapter]{article}` and the build breaks. Change `classoptions` and `toplevel` along with the class.

A `preamble:` key **replaces** the base header's preamble; `preamble+:` **appends** to it (section 21). Use the latter to keep the default and merely add packages.

1.2. Sharing configuration files between documents

`mapping.yaml` files and `setup:` header files exist to be shared across manuscripts, so when one cannot be found near the document, a set of **shared resource directories** is searched.

The order is as follows; the nearest match always wins.

1. An absolute path is used as given.
2. Near the document — `setup:` resolves beside the file that declares it, a front matter `mapping:` beside the top-level input file, and the CLI's `--setup`/`--config` relative to the working directory.
3. The `NDHMARKDOWN_RESOURCE_DIRECTORY` environment variable. It may list several directories, separated by the path separator (`;` on Windows, `:` elsewhere).
4. Conventional locations under the user's home directory — `~/Library/ndhmd4tex/resources` (macOS), `~/ndhmd4tex/resources`, `~/.config/ndhmd4tex/resources`.
5. A `resources/` directory beside the program itself, for shipping examples with a distribution.

```
export NDHMARKDOWN_RESOURCE_DIRECTORY=~/Documents/ndhmd-shared
python3 ndhmd4tex.py draft.md       # setup: basicpreamble.nmd is found there
```

- All three locations under (4) are searched rather than just the one for the current platform. Configuration is commonly synchronized across machines, and a directory that does not exist costs nothing to skip.
- When nothing is found, **every path tried is listed**. A feature with several search locations is unusable without that.

- `ndhmd-default.nmd` from 1.1 consults the same directories (working directory → program directory → resource directories → the copy bundled inside the executable).
- **`{{include}}` does not use this search.** Transclusion is document content rather than configuration, and belongs to the document's own directory.

Splitting `mapping.yaml` across several files

Front matter's `mapping:` accepts a **list**, not just a single file, the same way `setup:` does. This is for keeping a company- or team-wide set of rules in one file and a project-specific set in another, and using both together in a single document.

```yaml
mapping: [common.yaml, project-specific.yaml]
```

Merging follows two different rules depending on the key:

- **Every ordinary key except `extra_inline` follows exactly the same rule as `setup: [a, b]` — the later file wins.** If both `common.yaml` and `project-specific.yaml` define `headings:`, `project-specific.yaml`'s wins. This matches the override convention used throughout this dialect's front matter (later, more specific settings win).
- **`extra_inline` is the one exception — it's concatenated across every file instead.** Applying the "later wins" rule here would let `project-specific.yaml`'s `extra_inline` wipe out `common.yaml`'s entirely, defeating the whole point of splitting extension rules across files to use them together. So this key alone is combined rather than overwritten — rules defined in `common.yaml` and rules defined in `project-specific.yaml` both apply, without erasing each other.
- `environments:` (custom environments, section 22) needs none of this special treatment — it's a `{environment name: definition}` dict rather than a list to begin with, so files defining different names already coexist naturally, and files that redefine the same name merge field-by-field (`begin`/`end` and the like), so a partial override survives intact.
- `--config` (the CLI flag) only accepts a single file for now. Listing several files this way is only available through front matter's `mapping:`.
- If any file in the list can't be found, that's a hard error — it names which file was missing and lists every path that was tried.

Searching subdirectories too

If `mapping:`/`setup:` gives **a bare filename with no path** (not an explicit relative path like `deep/mapping.yaml`, just `mapping.yaml`), and it isn't found directly under (4) above, that directory is also searched **recursively**. This is for keeping `mapping.yaml`/setup `.nmd` files organized into per-purpose subfolders (`resources/work/`, `resources/personal/`, etc.) while still writing just `mapping: mapping.yaml` in the manuscript.

```
~/ndhmd4tex/resources/
├── deep/
│   ├── mapping.yaml          <- this one is picked
│   └── deeper/
│       └── mapping.yaml      <- this one loses out
```

- **"Found first" means "sitting in the shallowest directory."** If the same filename exists at both `deep/` and `deep/deeper/` as above, the shallower `deep/mapping.yaml` wins. Ties at the same depth are broken by alphabetical folder name, so the result is the same every time it runs.
- **An already-written path skips recursion entirely.** Writing `mapping: deep/deeper/mapping.yaml` is an explicit "look exactly here," so no other candidate gets slipped in via recursion.

- **A match near the document (near) always outranks recursion.** If the same filename sits right beside the document or in the working directory, that's settled before any recursive search inside the shared resource directories even begins.
- The error message for a failed lookup doesn't enumerate every path tried recursively — that would be unreadable for a deep or crowded tree. Instead it shows a wildcard like `resources/**/mapping.yaml`, meaning "somewhere under this directory."
- If it's unclear which of several same-named files won, `--search-resource` below lists everything found recursively too, in shallow-to-deep order.

1.2.1. `--search-resource` — checking where things are actually found

Rather than memorizing the search order above, you can just ask where something is actually found.

```
python3 ndhmd4tex.py --search-resource              # only the script/executable's own loca
python3 ndhmd4tex.py --search-resource article.nmd  # current directory + every resource di
```

- With no argument, it prints what `$0` refers to — the currently running script or executable's own path. Under a PyInstaller executable this is the exe's own path, not the temporary extraction directory.
- **It also shows where `ndhmd-default.nmd` actually comes from.** It walks the full search order from 1.1 (working directory → program directory → resource directories → the copy bundled inside the executable) and lists every one that is found, in that order. If more than one exists, **the first listed is the one actually used.** Even the bundled copy inside an exe is located precisely, so you can see which temporary directory it resolves to.
- Given a filename, it does **not stop at the first match**; it searches the current directory and every shared resource directory and lists every path found. This is for the moment when the same name exists in more than one place and it's unclear which one is actually being used — confirming that `setup:` or `mapping:` is quietly picking up a different file than intended. **Given a bare name with no path, it also searches the shared resource directories recursively**, listing same-named files in subdirectories too, in shallow-to-deep order — this shows exactly where the one `mapping:`/`setup:` actually picks (the shallowest one) sits relative to the others. **Naming `ndhmd-default.nmd` specifically** also walks the same search order as the no-argument form above (see 1.1) — this file is most commonly placed right beside the script/exe, a location distinct from the shared resource directories.
- If nothing is found, every path tried is printed and the exit status is 1.

1.2.2. `--guide` — opening a guide document directly

Place this guide, the quickstart, or similar documents (`.md`, `.pdf`) in `~/ndhmd4tex/guide` (also checked on macOS at `~/Library/ndhmd4tex/guide`, and via XDG at `~/.config/ndhmd4tex/guide` — the exact same three-tier structure as `resources/`, just applied to `guide/` instead), and a single command opens the PDF directly, the way `texdoc` does.

```
python3 ndhmd4tex.py --guide                                # ndhmd4tex_dialect_guide.pdf
python3 ndhmd4tex.py --guide=ndhmd4tex_dialect_guide_en     # the English edition (extension
```

- With no argument, it opens `ndhmd4tex_dialect_guide.pdf`.
- If there's no extension, `.pdf` is appended automatically (the same convention as `--default`'s `.nmd` auto-fill).
- The search order matches `resource_dirs()`: the `NDHMARKDOWN_GUIDE_DIRECTORY` environment variable → conventional locations under the user's home directory → a `guide/` folder beside the program itself.

- If nothing is found, every path tried is listed. Even when none of those directories exist at all, the conventional candidate paths are shown regardless — so there's still something telling you where to put the file.
- The PDF viewer is launched the same way `--pdf` and friends do after compiling (overridable with the `NDHMARKDOWN_PDF_VIEWER` environment variable).

1.3. `--lint` — checking for common markup mistakes

A handful of mistakes in this dialect **produce no error and simply yield something else**: the conversion succeeds, a PDF comes out, and a list has quietly collapsed into a paragraph or the footnotes have multiplied. Those are hard to spot by eye. `--lint` skips the conversion and looks only for those.

```
$ python3 ndhmd4tex.py manuscript.md --lint
manuscript.md:12: 오류: 리스트 마커가 문단에 흡수됐습니다. ...
manuscript.md:25: 경고: `[^ndh2024x]` 는 정의가 없어 ...
manuscript.md: 오류 2건, 경고 1건
```

(The diagnostics are in Korean: 오류 = error, 경고 = warning.)

The checks are not guesses — every one of them is a mistake actually made while writing this document.

Level	Check
error	A tab in the front matter indentation — the whole header is ignored
error	Broken YAML in the front matter — the whole header is ignored
error	A blank line inside a list item breaking the list
error	Body text glued under a heading with no blank line
error	`::: {name}` — a space between `:::` and `{`
error	An unclosed `:::` block
error	An unclosed code fence
error	A footnote definition identifier with illegal characters
error	A `{{include}}` file that is missing, or written without its extension
warning	An undefined `[^id]`, which silently becomes an inline footnote
warning	A space-surrounded `::` in prose, read as a description list (not for the braced form `{term} :: definition`)
warning	Something table-shaped whose second row is not an alignment separator
warning	A closing `:::` with nothing open

Code fences and inline code spans are skipped, so syntax quoted as an example is never mistaken for a real mistake. The exit status is **1** when any error is found, which makes it usable as a check step in a Makefile or script.

The first two matter most. When the YAML breaks, `class`, `setup` and `mapping` all stop taking effect while the conversion still succeeds, so nothing announces the problem. The usual cause is **a tab that came along when a preamble was pasted in from a LaTeX editor** — over there a tab is just whitespace and shows no sign of itself.

`{{include}}`d files are checked too (one level, as in section 20). For a book the master file is just a stack of `{{...}}` lines with all the prose in the included files, so without this there would be nothing left to check.

```
$ python3 ndhmd4tex.py book.md --lint
parts/ch1.md:6: 오류: 리스트 마커가 문단에 흡수됐습니다. ...
```

```
parts/ch2.md:4: 경고: `[^ndh2024x]` 는 정의가 없어 ...
book.md (+포함 2개): 오류 1건, 경고 1건
```

The files are not merged into one buffer first — that would make the line numbers useless. Each is checked on its own, except that **footnote definitions are pooled across all of them**: a reference in the master with its definition in an included file is entirely normal, and checking the files in isolation would report every one of those as undefined.

Linting never blocks conversion. Almost everything it reports converts perfectly well; whether to change it is the author's call.

1.4. `--pandoc` — hand only the body to pandoc

Use this when you want to write the body in **pandoc's markdown** instead of this dialect. Everything else stays with this script, which effectively turns it into a **pandoc LaTeX processor with transclusion**. Compiling and opening the viewer continues to work the usual way.

```
python3 ndhmd4tex.py book.md --pandoc --pdfxe
```

The order of operations is:

1. The front matter is split off and set aside. **Only the body goes to pandoc** — hand it the front matter and pandoc interprets it as its own metadata, which collides with the `preamble` and `setup:` machinery here.
2. `{{file}}` transclusion (section 20) is resolved **first**, producing one flat body. pandoc does not know that syntax, so it has to be merged beforehand. Front matter in the included files is stripped as usual.
3. The body is piped to pandoc on stdin and a LaTeX fragment comes back. `--standalone` is not used, so no `\documentclass` or `\begin{document}` appears.
4. The front matter set aside in step 1 builds the preamble around it. `class`, `classoptions`, `preamble`, `title`, `author`, `date` and `setup:` inheritance all behave exactly as they normally do.

`toplevel:` is forwarded to pandoc. If it is `part`, `chapter` or `section` it becomes `--top-level-division`; without it, `# Title` comes out as `\section` even under the `book` class. Anything below `subsection` has no pandoc equivalent and is ignored.

There are two ways to pass extra arguments to pandoc. If both are present, the front matter's come first and the CLI's are appended after.

```
# front matter
pandocoption: -f markdown-auto_identifiers --wrap=none
```

```
python3 ndhmd4tex.py book.md --pandoc="--listings --number-sections"
```

On the command line the `=` is **required**. Write `--pandoc "--listings"` with a space and argparse mistakes the value — which begins with `-` — for the next option.

Options that are ignored

This mode assumes the preamble and `\documentclass` come from the front matter. pandoc options that try to control them are dropped, with a warning.

Ignored	Why
`-s`, `--standalone`	pandoc emits a complete document with its own `\documentclass`
`--template`	same — supplying it turns standalone on
`-H`, `--include-in-header`	injects a preamble, and also turns standalone on
`-B`, `-A`, `--include-before-body`, `--include-after-body`	likewise turn standalone on
`-o`, `--output`	the result leaks into some other file
`-t`, `--to`, `-w`, `--write`	the output stops being LaTeX
`--pdf-engine`	only meaningful together with `-o`
`--self-contained`, `--embed-resources`	irrelevant to LaTeX output

Seeing `-H` and `-B` on that list may be a surprise. Their names suggest they would work in fragment mode, but pandoc **implicitly turns standalone on** when given them, and a second `\documentclass` then ends up nested inside this script's document.

Everything else passes through: `--toc`, `-N`/`--number-sections`, `-V`, `--listings`, `--metadata`, `--top-level-division`, `--wrap`, `-f`. Anything that belongs in the preamble goes in the front matter's `preamble:` key, not through pandoc.

`-f markdown-auto_identifiers` is worth considering. By default pandoc attaches a `\hypertarget{...}` and `\label{...}` to every heading; for CJK titles those labels come out as unreadable strings like `uxbcf8uxbb38`, and **the build breaks outright if `hyperref` is not in your `preamble`.** Turning identifiers off yields a clean `\chapter{Title}`.

In this mode **none of the dialect syntax described in this document works** — no `:::` environments, no `[[index]]`, no `[#label]`, no `showexam`, no Josa macros. The body is pandoc's entirely. Only the front matter and `{{file}}` survive.

1.4.1. `--mmd` — hand only the body to MultiMarkdown

Same shape as `--pandoc` (added in 0.20.2). The front matter (`---` YAML) is read by this script, `{{file}}` transclusion is resolved first into one flat body, and **only the body** is fed to `multimarkdown -t latex`. The LaTeX fragment that comes back is wrapped in the preamble built from the front matter. The executable is looked up on PATH as `multimarkdown`, then `mmd`.

```
python3 ndhmd4tex.py old.md --mmd --pdfxe
python3 ndhmd4tex.py old.md --mmd="--smart"
```

Extra arguments can also come from the `mmdoption:` key; when both exist the CLI part comes last. Attach CLI values with `=`. Options that change the output format or the snippet mode (`-t`/`--to`, `-o`/`--output`, `-f`/`--full`, `-s`/`--snippet`, `-b`/`--batch`, `-e`/`--extract`) are dropped with a warning. It cannot be combined with `--pandoc`.

Heading level — `toplevel` and `Base Header Level`. mmd sets the heading level only through `Base Header Level: N` in the metadata block at the top of the document. The script decides the value in this order:

1. `toplevel:` in the front matter (`part` / `chapter` / `section`) wins.
2. Otherwise `baseheaderlevel:` (an integer) in the front matter.
3. If neither exists and the body already starts with a metadata block holding `Base Header Level`, it is left alone. With nothing at all, mmd's default applies.

The mapping from `toplevel` to N is not hard-coded. Whether `#` becomes `\part` or `\chapter` can differ between mmd versions, so at start-up the installed mmd is asked once to convert `# … ######`

and N is computed from the commands that actually came out. If the probe fails, a warning is printed and the table `part=1, chapter=2, section=3` is used. When the body's own metadata block has a `Base Header Level` that disagrees with `toplevel`, a warning is printed and that line is rewritten to follow `toplevel`. If the block exists but lacks the line, one is appended to the block; if there is no block, a new one is put in front.

Things to know:

- mmd's `Title`, `Author` and `Date` are **not imported.** mmd does not fence metadata with `---`; any first line of the form `key: value` starts a block, so the script cannot tell whether such a line left in the body belongs to the front matter or to mmd. Put title and author in the front matter.
- mmd's `{{TOC}}` is a table-of-contents directive, not a file, so it is excluded from transclusion and passed on to mmd as is.
- If mmd emits a complete document instead of a fragment (judged only by the first meaningful line of the output being `\documentclass` or `\input`), only what lies inside `\begin{document}` is kept, with a warning. A literal `\begin{document}` inside a verbatim example in the body is not mistaken for one.
- If the front matter YAML is broken (for instance a tab after `key:`), the block is treated as body and mmd reads it itself. A `---`-fenced block at the top of the body that holds `Base Header Level` is found and handled by the rules above.
- mmd's LaTeX uses things like `\href`, `tabulary` and `longtable`; the packages they need are the front matter's `preamble:` to supply.
- Features of this dialect (`||`, grid tables, `showexam`, `notes:`, `:::`, ...) do not work in this mode; the body syntax is mmd's. `--syncinfo` and `--lint` are unavailable too. The provenance comment at the top of the `.tex` ends with `with multimarkdown syntax`.

1.5. `--syncinfo` — reverse search from PDF back to the source markdown

`--syncinfo` makes the conversion also write a `{output}.nmdsync` file alongside the `.tex`. Paired with `ndhmdsync.py` (a companion script shipped separately), clicking a spot in the compiled PDF can jump straight to the corresponding location in the original `.md` file in an editor.

```
PDF click --[SyncTeX]--> tex line --[.nmdsync]--> md line --[code --goto]--> VS Code
```

```
python3 ndhmd4tex.py book.md --syncinfo --pdfxe
```

- **No need to add the synctex option yourself.** `--syncinfo` appends it to the compile command automatically — `ksruncltex` (`compileoption:`) gets `--synctex=1` (two dashes, `ksruncltex`'s own convention), and `latexmk` (`latexmkoption:`) gets `-synctex=1` (one dash, `latexmk`'s actual syntax). If `compileoption:`/`latexmkoption:` already has `-synctex=...` or `--synctex=...` (either dash count), that value is respected and left untouched. `-synctex=1` solves "this spot in the PDF → this line in the .tex file" (a feature of the LaTeX engine itself). `--syncinfo` picks up from there with "this line in the .tex → this line in the original .md". Chain the two and you can go straight from the PDF to the markdown.
- Registering `ndhmdsync.py` as the "inverse search" command in a PDF viewer (SumatraPDF, Skim, Okular, etc.) completes this chain automatically. The default editor is VS Code and the default viewer is chosen automatically for the running OS, but both can be swapped for a different program (or pointed at a non-standard install location) via an `ndhmdsync.ini` config file — see `ndhmdsync.ini.example`. See `ndhmdsync.py --help` for usage.
- **A document with no `.nmdsync` but a `.synctex.gz`/`.synctex` file (i.e. plain LaTeX compiled directly, never touched by `ndhmd4tex.py`) can use the same inverse-search setup too.** When `ndhmdsync.py` can't find `.nmdsync`, it checks for `.synctex.gz`/`.synctex`; if one

exists, it gives up on tracing back to markdown and opens the `.tex` file itself at that line instead — via the command in `ndhmdsync.ini`'s `[TEXEDIT]` section if one is set (e.g. `texworks -p=%l "%f"`), or otherwise whatever program the OS has associated with `.tex` files (no line jump, but at least the file opens). Register the viewer's inverse-search command once, and it handles markdown-based and plain-LaTeX documents alike.

- **Precision is at the block level.** Which paragraph, list item, table, or environment a given source range came from is accurate; which exact word inside it is not tracked. That precision is exactly what this feature needs — landing near the right spot in the editor after a rough click in the PDF.
- **Documents using `{{file}}` are supported too.** Because this dialect only allows one level of file inclusion (no recursion) by design, the merged body's line origins can be tracked precisely — each `.nmdsync` entry carries its own source file, so `ndhmdsync.py` needed no changes at all; it was already built to handle a `.nmdsync` with more than one source file mixed in.
  - Nesting `{{...}}` two or more levels deep is left as literal text, exactly as this dialect's existing rule dictates (no recursive resolution) — the sync entry for that spot correctly points at the line where the nesting was attempted.
  - **A known limitation used to exist here, but it's now solved** by the `ndhmain:` front matter key (see below).
- **Cannot be combined with `--pandoc`.** Body conversion is handed off entirely to a pandoc subprocess, so there's no way for this script to know how many lines of output came from how many lines of input. Same outcome here: a warning, and conversion continues without a `.nmdsync`.
- Footnote **definition** lines (`[^id]: ...`) have no sync entry — their content is inserted at the point where they're **referenced**, not where they're defined, so the definition line itself corresponds to nothing in the output. Looking things up from the reference point is the more useful behavior anyway.

Running a forward search from `ndhmdsync.py` while an included sub-file itself is the open editor file will otherwise guess the wrong output `.tex`/`.pdf` path (it defaults to a name matching the sub-file, which is never compiled on its own — it's absorbed into the top-level file). Since a sub-file has no way of knowing which top-level document it gets pulled into, that relationship is spelled out directly in the sub-file's own front matter.

```
---
ndhmain: main.md
---
```

`ndhmd4tex.py` never reads this key at all (when a file is pulled in via `{{file}}`, its entire front matter is discarded, so this has zero effect on compilation). It's purely a hint `ndhmdsync.py` consults when guessing the default output path for a forward search. If the value is absent or points at the file itself (identity), the file is treated as its own top-level document as usual. Explicit `--tex`/`--pdf`/`--sync-file` always take precedence over this hint.

The value can be relative. In the common layout where `main.md` includes a file in a subfolder via `{{sub/ch1.md}}`, `ch1.md` can use a path relative to itself.

```
---
ndhmain: ../main.md
---
```

No matter which working directory `ndhmdsync.py` is run from (inside `sub/`, somewhere else entirely, or with an absolute path), this relative path is resolved against where `ch1.md` itself actually sits, so it always finds `main.md` (and the `main.nmdsync` next to it) correctly.

1.6. `--latexmk` — compile with a standard tool instead of ksruncltex

`--pdf`/`--pdfxe`/`--pdflua` compile through `ksruncltex`, a tool this script's author wrote. If you don't have `ksruncltex`, or you're already using `latexmk` and want to keep using it, use `--latexmk` instead.

```
python3 ndhmd4tex.py book.md --latexmk
```

With no value given, this runs `latexmk -xelatex -interaction=nonstopmode` by default (xelatex, since this dialect deals in CJK text).

`latexmk`'s right invocation depends on the situation — bibtex vs. biber, makeindex vs. xindy, whether to turn on synctex — in a way that can't be boiled down to one `compileoption:`-style line the way `ksruncltex` can. So this script doesn't try to guess that combination for you — it just takes `latexmk`'s own option string and passes it through unvalidated.

```
python3 ndhmd4tex.py book.md --latexmk="--synctex=1 --xelatex"
```

```
# put it in front matter so plain --latexmk applies it every time
latexmkoption: "-xelatex -synctex=1"
```

- On the CLI, it **must be attached with `=`** — same reason as `--pandoc`: a value starting with `-` would otherwise be mistaken by `argparse` for the next option.
- Front matter's `latexmkoption:` comes first, the CLI's `--latexmk="..."` value comes after (the same ordering rule as `pandocoption:`/`--pandoc`). The `-xelatex` default is only added when no option was given at all — give even one, and that default is left out entirely (so it can't collide with whatever engine you picked).
- `-interaction=nonstopmode` is always appended regardless of what else was given — without it, the `*latex` engine underneath hangs waiting for terminal input the moment something goes wrong.
- `latexmk`'s real syntax uses a single dash (`-xelatex`), but it also tolerates a double dash (`--xelatex`) — confirmed by actually testing it, a quirk of Perl's `Getopt::Long`. There is, however, **no such option for biber** — neither `-biber` nor `--biber` exists; `latexmk` rejects both as unknown. Biber use is auto-detected instead: if biblatex has produced a `.bcf` file, `latexmk` picks biber over bibtex on its own.
- A bad option makes `latexmk` itself report a clear error — this script doesn't validate or patch anything on your behalf. Same philosophy as `ksruncltex`'s `compileoption:`.
- `--pdf`/`--pdfxe`/`--pdflua`/`--latexmk` are mutually exclusive (can't combine them).

If front matter has `latexmkoption:` but not `compileoption:`, `--pdf`/`--pdfxe`/`--pdflua` alone will invoke `latexmk` automatically — no need to also pass `--latexmk`. The document itself now carries the meaning "I compile with `latexmk`" — the engine follows whatever was picked on the command line (`--pdfxe` → `-pdfxe`, `--pdf` → `-pdf`, `--pdflua` → `-pdflua`), with `latexmkoption:`'s value appended after it.

```
latexmkoption: "-synctex=1"
```

```
python3 ndhmd4tex.py book.md --pdfxe
```

```
# with the front matter above, this actually runs:
#   latexmk -pdfxe -synctex=1 -interaction=nonstopmode book.tex
```

The `-synctex=1` examples in this section are still useful **when you want SyncTeX on its own, without `--syncinfo`**. If you're also using `--syncinfo`, you don't need to add it by hand — as described in 1.5, it gets appended automatically (or left as-is if you already set it).

If `compileoption:` is present (even with an empty value — **merely having the key counts**), this auto-switch doesn't happen — that document is still treated as using `ksruncltex`. A document with neither key keeps working exactly as it always has, through `ksruncltex`. Explicitly typing `--latexmk` (with or without a value) behaves exactly as described above, independent of this auto-switch.

---

2. Front matter

Put a YAML block wrapped in `---` at the very top of the file.

```
---
title: "Document Title"
author: "Author Name"
date: "\today"          # default is \today if omitted; an empty string omits \date entirely
class: article           # default is book
classoption: "11pt,a4paper"   # classoptions is accepted as a synonym
toplevel: section        # see "3. Headings" below
mapping: mymap.yaml       # can be set here instead of --config
setup: mymaster.md        # inherit the header from another file (see section 21)
beforestart: |            # placed ahead of \documentclass
  \DocumentMetadata{lang=ko}
theme: dark               # unknown key -> \def\NMtheme{dark}
compileoption: "-shell-escape"   # extra args for --pdf-family compilation
preamble: |
  \usepackage{kotex}
  \usepackage{amsmath}
---
```

`author` list example:

```
author:
  - Nova de Hi
  - A. U. Thor^[thor's affiliation]
```

```
\author{Nova de Hi \and A. U. Thor\thanks{thor's affiliation}}
```

- `title` / `author` / `date`, when present, become `\title{}` / `\author{}` / `\date{}` respectively. A missing key means the command is simply omitted. For `date` that means no `\date` is emitted, so the class default (normally `\today`) applies; **a key that is present but empty** (`date:` or `date: ""`) emits `\date{}`, which clears the date.
- The first line of the generated `.tex` is a provenance comment such as `% converted from ndhmd by ndhmd4tex version 0.19.1` (with `with pandoc syntax` appended under `--pandoc`; no date or path, so the same input always gives the same output). It is not emitted with `--default NONE` (body-only output).
- **`author` also accepts a list.** Entries are joined with `\and`, following the LaTeX convention of listing several authors as `\author{A \and B}`. A `^[...]` inside an entry (the same syntax as the inline footnotes of section 16) turns into `\thanks{...}` rather than `\footnote{...}` — the usual LaTeX convention for affiliations, funding notes, and the like on the title page. Like `title`, `author` is treated as raw text — nothing beyond this `^[...]` substitution and the `\and`-joining is processed.
- `preamble` is inserted raw right after `\documentclass` and before `\begin{document}`. Put any packages/definitions you need here (`kotex`, `amsmath`, `\newtheorem`, etc.).
- `beforestart` goes **ahead of `\documentclass`**, in the same format as `preamble`. It is the place for commands that only mean anything before the class is loaded, such as `\DocumentMetadata` or `\PassOptionsToPackage`. `beforestart+` inherits and appends, as elsewhere.
- **The contents of `preamble` and `beforestart` are emitted untouched** — no escaping, no inline processing — so expl3 syntax can be written directly.

```
  preamble: |
    \ExplSyntaxOn
```

```
\tl_set:Nn \l_tmpa_tl { my string }
\ExplSyntaxOff
```

Underscores turn into `\_` only in the **body**. For expl3 code there, wrap it in the inline raw span of 7.1 or a `raw` code fence from section 10 — `` `\ExplSyntaxOn\tl_use:N \l_tmpa_tl\ExplSyntaxOff`{=latex} ``.

- **`graphicspath` appends two lines to the very end of `preamble`.**

```
graphicspath: ["./","figures/"]
```

```
\IfPackageLoadedTF{graphicx}{}{\usepackage{graphicx}}
\graphicspath{{./}{figures/}}
```

If `graphicx` is already loaded (whether the author's own `preamble` called it directly, or some other package pulled it in), it isn't loaded again — `\IfPackageLoadedTF` is a LaTeX2e kernel command, no extra package needed. A single string works too (`graphicspath: "figures/"`). The path strings themselves pass through unprocessed, wrapped in `{}` as-is — the same stance as the `raw=` family of attributes: this dialect doesn't try to second-guess LaTeX's own path syntax.

2.1. Backslashes in YAML values

`\` is an escape character in YAML **only inside quotes**, which gives the same value more than one spelling.

Written	Result
`date: \today`	`\today` — unquoted, taken literally
`date: "\\today"`	`\today` — doubled inside double quotes
`date: '\today'`	`\today` — single quotes need no doubling
`date: "\today"`	**a tab character followed by `oday`**

The last row is the trap. Inside double quotes `\t` is a tab and `\n` a newline, so `\today`, `\newpage` and `\textbf` quietly become something else, with no error to warn you.

Not quoting is safest. If a value contains `:` or `#` and needs quoting, use **single quotes**. For anything multi-line use a block scalar (`preamble: |`), inside which escaping never applies.

2.2. Unknown keys become macros

A key the script does not use is passed to the preamble as `\def\NM<key>{value}`, placed before `preamble` so that the preamble can read it.

```
---
theme: dark
mysize: 42
---
```

```
\def\NMtheme{dark}
\def\NMmysize{42}
```

- The `NM` prefix is configurable as `metadata_macro_prefix` in `mapping.yaml`, and an empty string disables the feature. Passing these through without a prefix, as multimarkdown does (`\def\theme{dark}`), turns ordinary words into control sequence names and can silently override a command from the class or a package.
- **Keys are lowercased** to form the macro name, so `Theme: dark` also yields `\NMtheme`. LaTeX macro names are case-sensitive, and this way the preamble can spell it one way no matter how the manuscript wrote it. If two keys collide once lowercased, the later one wins and a warning is printed.

- A name such as `Class:` that differs from a **known key only in case** is not used as that key; only a `\NMclass` macro appears. Nothing errors, so a warning is printed.
- **Only keys spelled with letters** are passed. Names containing digits, underscores or hyphens — and non-ASCII names — cannot form a control sequence and are skipped silently.
- Only scalar values (string, number, boolean) are passed; lists and mappings are skipped.
- `class` / `classoption` build `\documentclass[options]{class}`.
- If repeating this header across documents gets tedious, factor the shared part out into a separate file and inherit it with `setup:`. See **section 21**.

2.3. `maketitle` / `tocloft` — auto-inserting title and TOC-style lists

Automatically inserts `\maketitle`, `\tableofcontents`, `\listoffigures`, and `\listoftables` right after `\begin{document}` (the way pandoc does).

```
maketitle: true
tocloft:
  toc: true
  lof: true
  lot: true
  asterisk: true

\begin{document}

\maketitle
\tableofcontents*
\listoffigures*
\listoftables*

body...
```

- `maketitle: true` alone produces just the `\maketitle` line. `tocloft:` turns on whichever of `toc`/`lof`/`lot` are wanted underneath it — any left off simply don't appear.
- The order is always fixed: `\maketitle` → `\tableofcontents` → `\listoffigures` → `\listoftables`.
- **`asterisk: true` only means something when `class` is `memoir` or `oblivoir`.** Those two classes give `\tableofcontents*` its real meaning — "don't list this table of contents as an entry of itself" (a feature of memoir's `\newlistof` macro). Standard classes like `article`/`report`/`book` have no starred variant of these commands at all, so emitting `\tableofcontents*` there **still compiles, but the `*` isn't consumed as part of the command — it's typeset as a literal character on the TOC page** (a pitfall confirmed by actually compiling and inspecting the output). Using `asterisk: true` outside those two classes prints a warning but doesn't block the build — a custom class might define its own starred variant.
- Both keys are inherited through `setup:` like any other front matter key.

This feature is convenient for a casual piece of writing or a single simple article, but can be a trap for book-length work. `\maketitle` just stamps out one plain title page — oblivoir/memoir separately provide a `titlingpage` environment for a much more deliberate title page, and this feature never touches that. A manuscript running to book length usually needs each of the following decided by hand, per book:

- `\frontmatter`/`\mainmatter`/`\backmatter` — which section you're in changes the page-numbering scheme (roman vs. arabic) and chapter numbering. This feature inserts none of these three.

- `\pagestyle`, `\chapterstyle` — page and chapter-heading design that's chosen per book, with no sensible default to auto-fill.

These belong in front matter's `preamble:` or `beforestart:`, written out by hand. `maketitle`/`tocloft` do exactly what their names say and nothing more — one title page and a handful of TOC-style lists dropped in at the front of the document.

---

3. Headings

```
# Level 1 title
## Level 2 title
### Level 3 title {unnumbered}
#### Level 4 title {label=sec:foo}
##### Level 5
###### Level 6
```

- Default mapping (levels 1–6): `chapter`, `section`, `subsection`, `subsubsection`, `paragraph`, `subparagraph`. **The `article` class has no `\chapter`, so when using `article` you must set `toplevel: section` (or `part`) in the front matter so that level 1 starts at `\section`.**
- `{...}` attributes after a heading:
  - `unnumbered` or `*` → an unnumbered heading (`\section*`, etc.)
  - `label=name` or `#name` → automatically inserts `\label{name}`
  - `toc="short title"` → `\section[short title]{long title}`. Use it when the table of contents and running headers should carry the short form while the body keeps the long one. Combined with `unnumbered` it is silently ignored, since the starred form has no bracket argument at all — rather than error out on a combination LaTeX would not accept anyway, the unnumbered title is simply kept as written.
  - Example: `## Introduction {unnumbered, label=sec:intro}`
  - Example: `# A rather long section title, shortened in the TOC {toc="Short title"}`
- The entire heading line must match `HEADING_RE`, and a heading must be its own single-line block (separated by blank lines on both sides).

---

4. Paragraphs and line breaks

Within the same paragraph, lines are joined with a space into a single sentence (a plain line break by itself has no effect on the output). Leaving **two trailing spaces at the end of a line** inserts a LaTeX forced line break (`\\`) at that point.

```
This sentence is written across
multiple lines but gets joined into one paragraph.

This line ends with two trailing spaces.
So the line breaks here, forced.
```

Blank lines separate paragraphs.

4.1. || — joining without a paragraph break

In Korean, a paragraph that starts with a particle (라고 말하기 시작했다.), or one that continues after a list or a display, must be set **without an indent**. Instead of writing `\noindent` by hand, put a line containing only `||`.

```
"A short utterance."
||
라고 말하기 시작했다.

A paragraph ends here.
||
+ this is a sample
||
The continuing paragraph
```

"A short utterance."라고 말하기 시작했다.

A paragraph ends here.

- this is a sample

The continuing paragraph

- `||` is **a blank line as far as structure goes**: it splits its neighbours into separate blocks, so a `+ item` right after `||` is recognised as a list without a blank line (chapter 11).
- In the **output**, however, no blank line is emitted, so LaTeX reads the two blocks as one paragraph. After a list or a display this is why no indent appears.
- If **both** neighbours are plain paragraphs, a `%` is added to the end of the first, so the end-of-line space does not turn `''`라고 into `'' `라고. The feature is meant for Korean joins; there is no variant that inserts a space.
- **A blank line before `||`** puts `\\` instead of `%` at the end of the preceding paragraph, breaking the line there (only when both neighbours are plain paragraphs). Use it instead of writing `text\\` by hand. A blank line *after* `||` has no meaning. If the following paragraph starts with `[` or `*`, `{}` is appended so `\\` does not take it as an optional argument. When a neighbour is not a plain paragraph (list, quote, heading…), the blocks are simply joined, with no `\\`, blank line or not. An example follows the list.
- `--lint` warns about a `||` at the very start or end of the document and about consecutive ones.
- `||` inside a code fence is literal text. A line consisting only of `||` inside a pipe table splits the table there.
- It also works inside `:::` environments.
- This is **not** "new paragraph, just without the indent": it merges into one paragraph. To end the paragraph and only drop the indent, keep writing `\noindent`.

A blank line before `||` breaks the line.

```
First line

||
Second line
```

First line

Second line

---

5. Inline formatting

Syntax	Result
`***bold italic***`	`\textbf{\emph{...}}`
`**bold**`	`\textbf{...}`
`*italic*`	`\emph{...}`
`~~strikethrough~~`	`\sout{...}`
`^^dot emphasis^^`	`\dotemph{...}` (traditional "emphasis dots" used in Korean typesetting; requires `\dotemph` to be defined in `preamble`)
`` `inline code` ``	`\texttt{...}` (contents are auto-escaped)

Text wrapped in `**`/`*`/`~~` only goes through `escape_text` and is *not* recursively reprocessed (so you can't nest links, footnotes, etc. inside bold text). The one exception is that **smart-quote conversion runs before** this formatting step, so quotes inside bold text — e.g. `**"a quote"**` — are still converted correctly.

5.1. Adding rules — `extra_inline`

The five rules above live in `inline` in `mapping.yaml`, and that key is a list, so touching it **replaces the whole set**. Copying all five over just to add one is tedious and goes stale when the defaults change. `extra_inline` leaves the defaults alone and inserts your rules **in front of them**.

```yaml
extra_inline:
  - name: highlight
    pattern: '==(.+?)=='
    template: '\hl{%s}'
```

- **In front, not behind.** Rules apply top to bottom, so longer delimiters must come first. Append a `***...***` rule at the end and the earlier `**...**` consumes the text before it is ever reached.
- The flip side: a delimiter that is shorter than, or overlaps, a default one will shadow that default when placed in front — put `*...*` first and `**bold**` breaks. Use `extra_inline` for genuinely new delimiters; override `inline` itself to change an existing one.
- Order within `extra_inline` is preserved too, so list the longer of any overlapping pair first.

Rules taking more than one argument

Capture groups are passed to the template **in full**, so simply use several `%s`.

```yaml
extra_inline:
  - name: dialog
    pattern: '\*\[(.+?)\]\[(.+?)\]'
    template: '\Dialog{%s}{%s}'
  - name: dialog_short
    pattern: '\*\[(.+?)\]'
    template: '\Dialog{%s}'
```

```
*[Chunhyang][angrily] Where have you been?

\Dialog{Chunhyang}{angrily} Where have you been?
```

The two-argument rule must come **before** the one-argument one; reversed, it matches only `*[Chunhyang]` and leaves `[angrily]` as plain text.

Each group is escaped separately, so a `%` or `&` inside an argument is handled for you. A group that did not participate in the match becomes an empty string, and if the number of groups does not match the number of `%s`, a warning is printed and the original text is left untouched.

---

6. Quotes (smart-quote conversion)

LaTeX uses different characters for opening and closing quotes. This script automatically converts straight quotes (`"`, `'`).

Kind	Opening	Closing	How it's decided
Double quote `"`	``` `` ``` (two backticks)	`''` (two apostrophes)	see below
Single quote `'`	`` ` `` (one backtick)	`'` (unchanged)	see below

How it decides: only a quote character at the very start of the string, or immediately after a space / `(` / `[` / `{` / `*` / `~`, is treated as "opening" and converted first. Every other remaining quote character is then treated as "closing" and converted in one pass. **What follows a closing quote is never checked.** This matters specifically for Korean text, where a grammatical particle very often attaches directly to the closing quote with no space in between (e.g. `"quote"라고`, where 라고 is a Korean particle meaning roughly "[he] said"). Pandoc's usual approach — deciding the closing position by checking whether it's followed by a space or punctuation — simply doesn't work for that pattern.

```
He said "hello."
→ He said ``hello.''

("quote"라고 - a Korean particle attaches directly, no space)
→ ``quote''라고   (the closing quote is still recognized correctly)

**"an emphasized quote"** is converted correctly too.
→ \textbf{``an emphasized quote''}
```

Escaping: if you want a straight quote that should *not* be smart-converted, write `\"` or `\'` — the backslash is stripped and the literal character (`"` or `'`) is emitted as-is.

```
He wanted the \"real\" straight quotes.
→ He wanted the "real" straight quotes.
```

Quotes in the following contexts are unaffected by smart-quote conversion (they're already protected as placeholders by that point): inline code, inline/display math, raw macros, a figure/link's `{attr="value"}` or `(url "title")`, and any YAML value in the front matter.

6.0.1. `quotes` — choosing the output characters

The table above shows the default, `quotes: tex`. The `quotes` key in the mapping (`mapping:`, `mappingrule:`, `--config`) changes it (0.20.4).

Value	Double quotes	Single quotes
`tex` (default)	``` `` '' ```	`` ` ' ``
`unicode`	U+201C " U+201D "	U+2018 ‘ U+2019 ’
4 characters `"「 」『 』"`	first two (open, close)	last two (open, close)
2 characters `"《 》"`	those two	left as in `tex`
list `["<<", ">>"]`	for multi-character marks, a list of the same length	(last two of 4 items)

`unicode` is shorthand for the four characters `""‘’`. The open/close decision, the escapes (`\"`, `\'`) and the handling of quotes in code and math are the same as with `tex`; only the substituted characters change, and they are protected from later stages. An invalid value produces a warning and falls back to `tex`. It does not apply to `--pandoc` / `--mmd` (pandoc's smart extension and mmd's `--smart` handle that).

- Unicode quotes appear when the font in use has the glyph, under xelatex/lualatex. pdflatex needs `inputenc`.
- The closing single-quote character is also what an apostrophe becomes. With `unicode` that is U+2019, which is correct, but with 「 」『 』 you get `don』t`; protect such places with `\'` or write `\textquoteright{}`.

6.1. Ellipsis (...)

Three periods become `\ldots{}`. Set side by side, plain periods are spaced too tightly and are simply wrong typographically.

```
Three dots... like this. Four dots.... give ellipsis plus period.

Three dots\ldots{} like this. Four dots\ldots{}. give ellipsis plus period.
```

Runs of four or more are consumed three at a time from the left.

- **The trailing `{}` is not decoration; it is required.** Where Hangul syllables are letters (catcode 11), emitting `\ldots이렇게` makes TeX read the whole thing as one control sequence name and raise `Undefined control sequence`. The `{}` breaks that boundary — the same story as the macro name terminator in section 7.
- The command is configurable through `ellipsis` in `mapping.yaml`. Korean typesetting in particular needs this, since the shape and spacing of an ellipsis vary considerably from font to font.

```yaml
ellipsis: '\dots{}'          # amsmath; spacing adapts to context
ellipsis: '\textellipsis{}'  # the text-mode ellipsis
ellipsis: '……'               # two Unicode ellipses (traditional CJK six dots)
ellipsis: ''                 # no conversion; leave the three periods alone
```

- Not converted: code spans (`` `1...5` ``), code fences, math, an existing `\ldots`, link URLs, index terms. All of those are already protected by the time this rule runs.
- Note though that the **display text** of `[[shown...|term]]` is treated as raw along with the index term and is therefore left alone (section 16). Write the command yourself if you need an ellipsis there.
- A Unicode … typed directly is passed through untouched.

---

7. Inline math / macros / raw LaTeX

```
Inline math: $x^2 + y^2 = 1$ or \(x^2+y^2=1\)

Raw macros pass through untouched: \textbf{a hand-written macro} \cite[p.10]{key}

Short escape sequences: \\ \, \; \% \& \# \_ \{ \}
```

- `$...$` (no line breaks allowed) or `\(...\)` is protected as inline math and passed through unchanged.

- A macro of the form `\command[optional]{arg}{arg}...` is passed through **completely untouched** (one level of nested braces is supported). In other words, it's safe to mix raw LaTeX directly into the markdown whenever you need to.
- `\\`, `\,`, `\;`, `\%`, `\&`, `\#`, `\_`, `\{`, `\}` are recognized as short escape sequences and pass through as-is.
- **Korean particle (Josa) macros**: `\은`, `\는`, `\이`, `\가`, `\을`, `\를`, `\와`, `\과`, `\으로`, `\로`, and `\<` are recognized as fixed literal tokens and pass through unchanged (book editors frequently use macros that auto-select the correct Korean particle). Only exactly these strings are recognized; a Hangul word immediately following, with no space, is *not* greedily absorbed into the macro name. See the example below.
- **The `{}` name terminator**: append an empty brace group to the name and it passes through as a macro even if it is not in the literal list — `\이라{}고`, `\에서{}부터`. This is TeX's standard idiom for ending a control sequence without inserting a space; it already worked for ASCII names (`\LaTeX{}의`) and now covers Hangul names too. The same rule admits Hangul-named macros that take arguments, e.g. `\강조{중요}`. See below.
- A macro name may be followed by **several optional-argument brackets** — natbib's `\citep[see][p.~33]{ke` being the standard case. Brackets that come *after* a brace group are still not recognized (`\raisebox{2pt}[0pt][` use the inline raw span of 7.1 for those.
- For raw LaTeX fragments outside that list, use the inline raw span of 7.1.
- This list of literal tokens comes from `cfg["literal_macros"]` (default is the list above) and `cfg["extra_macros"]` (default: empty list), and both **can be customized via `mapping.yaml`**. If you just want to add a token or two for your project (leaving the default list intact), use `extra_macros`. See the example below.

Example: preventing greedy matching of particle macros

```
값\은 이렇게 된다. 값\은뭐라고말하기어렵다.

값\은 이렇게 된다. 값\은뭐라고말하기어렵다.
```

(the trailing word "뭐라고말하기어렵다" is *not* swallowed into the macro name.) `\으로` is checked before `\로`, so the longer token always wins. Any particle not in this list (e.g. `\도`) is treated as plain text and gets the ordinary auto-escaping described in section 18 (its backslash becomes `\textbackslash{}`).

Why `\라` and `\이라` are not in the defaults:

These particles rarely end a sentence on their own; in practice an auxiliary particle almost always follows — `\이라고`, `\이라는`, `\이라도`, `\이라서`. A literal-token list only ever recognizes the exact string, so those inflected forms cannot be covered by the rule.

The important part is that **the two fail in different ways.**

Manuscript	Result	Problem
`값\라`	`값\textbackslash{}라`	visibly broken, so you find it immediately
`값\이라`	`값\이라`	**looks fine and is wrong**

`\이라` is the dangerous one. `\이` is a literal token, so it matches first and 라 is left as plain text. The output string looks identical, but LaTeX reads it as "the `\이` particle, followed by the letter 라". When the preceding word ends in a vowel, `\이` expands to 가 and you get 가라 rather than 이라. No error, no warning.

The fix: end the name with `{}`.

```
값\이라{}고 하고, 값\이라{}는 것, 값\이라{}도 그렇다.

값\이라{}고 하고, 값\이라{}는 것, 값\이라{}도 그렇다.
```

The empty brace group declares "the macro name ends here", so the script never has to guess. `\이라` passes through even though it is not in the literal list, and `\이` cannot eat it. However many inflected forms you use, none of them needs registering.

This is not merely a convenience for the converter. **LaTeX needs it too.** In a setup where Hangul syllables are letters (catcode 11), writing `\이라고` makes TeX read the whole thing as one control sequence named `이라고` and raise `Undefined control sequence`. The `{}` breaks that boundary. So the manuscript notation and the output are identical, and both need it for the same reason.

The same rule covers Hangul-named macros taking arguments:

```
\강조{중요한 말}, \색{red}{글자}, \인용[p.10]{책}
```

The crucial part is that **at least one brace group is required.** Without it, there is no way to tell where the name ends in something like `\에서부터`, and the greedy-matching problem that motivated the literal list in the first place comes straight back.

If you use only a handful of inflected forms and would rather not type `{}` each time, registering them in `extra_macros` also works. Sorting is automatic, so order does not matter.

```
extra_macros: ['\이라고', '\라고', '\이라는', '\라는', '\이라', '\라']
```

Example: adding tokens via `extra_macros`

```yaml
# mapping.yaml - add \>, \| as raw macros
extra_macros:
  - "\\>"
  - "\\|"
```

If you want to change the default particle-macro list itself (rare), override `literal_macros` wholesale (list-valued keys get fully replaced on override — see section 19). Either way, prefix collisions between newly added tokens (e.g. registering both `\ab` and `\abc`) are handled automatically — the regex is rebuilt sorted longest-first every time, so the longer token always wins regardless of registration order. Note that this sorting only operates within the combined `literal_macros` + `extra_macros` list. A longer form that is not in the list at all (like `\이라` above) gets no protection and is eaten by the shorter token.

7.1. Inline raw spans — LaTeX in the middle of a sentence

The macro pass-through above only fires for things shaped like `\command[optional]{arg}`. Anything outside that shape is treated as ordinary text and gets the auto-escaping of section 18. When you need to drop such a fragment into the middle of a sentence, the escape hatch is an inline raw span — Pandoc's raw-attribute syntax, written as a code span with a format name attached.

```
Brace group: this is `{\bfseries emphasis}`{=latex} here.
Two levels of nesting: `\textbf{a \emph{b \textsc{c}}}`{=latex}
Optional arg after a brace: `\raisebox{2pt}[0pt][0pt]{lifted}`{=latex}
Math inside an argument: `\fbox{$x^2$}`{=latex}
```

The closing backtick and the opening brace must be **adjacent**. Put a space between them and you get an ordinary code span.

Those four are the representative cases the macro pattern cannot handle. Written without the span, they break like this:

Written plainly	What you get
`{\bfseries emphasis}`	a brace group is not a macro, so the braces get escaped
`\textbf{a \emph{b \textsc{c}}}`	only one level of nested braces is recognized, so the tail drifts
`\raisebox{2pt}[0pt][0pt]{lifted}`	an optional argument is only recognized right after the macro name
`\fbox{$x^2$}`	the `$` and `^` inside the argument get escaped

The accepted format names are `latex`, `tex` and `raw` (configurable via `raw_inline_formats`). **A span aimed at any other format is dropped entirely** — the same behaviour as Pandoc, since there is nowhere to put it in LaTeX output.

```
before`<br>`{=html}after
```

```
beforeafter
```

For raw LaTeX at the block level, use the `raw` / `tex` / `latex` code fences from section 10 instead. This span is for the **middle of a sentence**, where you cannot afford to break the paragraph.

7.2. The tilde (~) policy

Both ~ and \~ **pass through to the LaTeX source exactly as written.** The converter reserves the slot; whether it means a non-breaking space or a tilde character is LaTeX's decision.

Manuscript	Output
`Figure~1`	`Figure~1`
`lives in \~/bin`	`lives in \~/bin`
`steps 1\~6`	`steps 1\~6`

There is a reason for this. In LaTeX ~ is the non-breaking space (`\nobreakspace`) and \~ is an accent command — both are things the **LaTeX side is free to redefine**. Korean typesetting in particular sometimes uses a package that turns \~ into a tilde character when a digit or Hangul syllable follows. If the converter froze \~ into `\textasciitilde{}` up front, it would remove the very hook such a package needs. So the meaning is left to LaTeX.

For \~ to survive, **the preamble has to define it.** In stock LaTeX with no package loaded, \~ places a tilde accent over the next character, so `1\~6` typesets as 1̃6 and `\~/bin` as `/bin` — the tilde all but disappears. In a document with no such definition, either add one (`\newcommand{\~}{\textasciitilde{}}`) or write `\textasciitilde{}` in the manuscript directly.

`wavedash` — using \~ as a wave dash

The default behavior above (`wavedash: default`) leaves both ~ and \~ to LaTeX. If you want \~ in Korean text such as "13장" to come out as a wave dash directly, set `wavedash` in the mapping. For any value other than `default`, **~ stays a nobreak space and only \~ is replaced by this value.**

```yaml
wavedash: FF5E              # Unicode code point (hex): ～ (fullwidth tilde)
wavedash: "U+301C"          # a U+ prefix works too: 〜 (WAVE DASH)
wavedash: "～"              # a literal single character
wavedash: \koreanWaveDash   # a LaTeX macro (`{}` is appended automatically)
wavedash: default           # the default - the old behavior unchanged
```

- Quote code points or use the `U+` prefix. A bare number such as `3030` is read by YAML as an integer but interpreted as hex digits, while `0x3030` is turned into a decimal integer by YAML and ends up as the wrong character.
- A macro must be defined by you (for example in `preamble:`). The output is `\koreanWaveDash{}`.
- An unrecognized value prints a warning and falls back to `default`.
- It applies to text the converter processes: body, headings, footnotes, table cells. A ~ inside an inline code span is always `\textasciitilde{}` (see the exception below), and front matter values (`title:` etc.) are raw LaTeX, so \~ there is not replaced.
- A Unicode character needs a font that has the glyph (and input-encoding setup under `pdflatex`); if unsure, a macro is the safer choice.

`tilde` — the `~` and `^` dialect

The default (`tilde: latex`) is what is described above: `~` is left to LaTeX and `^` becomes `\textasciicircum{}`. For manuscripts imported from other Markdown dialects, the mapping can select one.

`tilde:`	`~`	`^`
`latex` (default)	passed through (nobreak space)	`\textasciicircum{}`
`pandoc`	`\textasciitilde{}`; `~x~` is `\textsubscript{x}`	`^x^` is `\textsuperscript{x}`
`multimarkdown` (`mmd`)	`a~b` is `a\textsubscript{b}`, as is `~x~`; a `~` with nothing after it is `\ensuremath{\sim}`	`a^b` and `^x^` are `\textsuperscript{}`

- Closed forms (`~x~`, `^x^`) must contain no whitespace. The open forms of multimarkdown (`a~b`, `a^b`) run over the following run of letters/digits (up to whitespace or punctuation).
- `\~` follows the `wavedash` rule in every mode, so in multimarkdown mode `1~3` (a subscript) is avoided by writing `1\~3`.
- `~~strike~~`, `^^dots^^`, `^[note]`, code spans, math and `\macro{…}` (including its arguments) are handled and protected first in every mode. A bare URL not wrapped as a link has its `~` turned into `\textasciitilde{}` (pandoc) or a subscript (multimarkdown); use `<…>` or link syntax.
- `\textsubscript` / `\ensuremath` in headings may raise hyperref bookmark warnings.
- The key is ignored under `--pandoc` (real pandoc does its own handling); the value `pandoc` and the option `--pandoc` are unrelated.
- Unknown values warn and fall back to `latex`.

The one exception: inline code spans

A `~` inside an inline code span is always escaped to `\textasciitilde{}`.

```
`rm -rf ~/tmp`
```

```
\texttt{rm -rf \textasciitilde{}/tmp}
```

A code span is not LaTeX the author wrote; it is a place that **shows text literally**. Left raw, the `~` becomes a non-breaking space and the command renders as `rm -rf /tmp` with the tilde evaporated. Every other special character — `_`, `$`, `&`, `{` — is escaped here too, so exempting `~` alone would be the odd choice. If you disagree, delete the `_with_tilde()` call in the script. Code fences are verbatim/minted and were never touched.

Interaction with inline formatting rules

Rules that use `~` as a delimiter, such as strikethrough `~~...~~`, keep working.

A tilde escaped as `\~` does not count as a delimiter: `a\~\~b\~\~c` comes out as `a\~\~b\~\~c`, not a strikethrough.

This works because the point where `\~` is protected is pinned from both sides. It must come after smart-quote conversion (so the double quote in `\~"quoted"` still opens correctly) and before the inline rules (so an escaped tilde never becomes a delimiter).

Restoring the old behaviour

Put `~` back into `escape_chars` in `mapping.yaml` and every bare `~` becomes `\textasciitilde{}` again. It is a list-type key, so an override replaces the whole list — spell out the other characters too (see section 19). Note that `\~` passes through regardless of this setting.

```
escape_chars: ["\\", "&", "%", "#", "_", "$", "{", "}", "~", "^"]
```

---

8. Display math / raw environments (protected at the block stage)

The following are protected as a single opaque unit **before the block parser even runs**, so nothing inside them — no escaping, no quote conversion, nothing — is ever touched; it's copied to the output verbatim.

```
$$
\int_a^b f(x)\,dx = F(b) - F(a)
$$

\[
E = mc^2
\]

\begin{equation}
a^2 + b^2 = c^2
\end{equation}

\begin{align}
x &= 1 \\
y &= 2
\end{align}
```

Supported raw environments: `equation*?`, `align*?`, `gather*?`, `multline*?`, `eqnarray*?` (math), and `tabular`, `tabularx`, `tabulary`, `tblr`, `longtblr` (hand-written tables). The `\begin{...}` and `\end{...}` names must match exactly; anything in between (matched non-greedily) is treated as one block, even across other content.

---

9. Comments (<!-- -->)

Just like Pandoc: everything from `<!--` up to the first `-->` that follows it is removed entirely (nesting is not supported, matching the HTML standard — an inner `<!--` is ignored, and the comment closes at the very next `-->`). Comments can span multiple lines, and anything inside them — headings, lists, math, inline formatting — is **never interpreted at all**; it's simply deleted.

```
This sentence stays. <!-- this part disappears --> This sentence stays too.

<!--
This entire block disappears.

## This heading disappears too

- This list disappears too
-->

Only this sentence remains.
```

A `<!-- -->` shown inside a fenced code block or inline-code backticks is *not* treated as a real comment and is preserved as literal syntax (for inline code, this is only safe when the backticks sit immediately next to `<!--`/`-->`, for the same reason described in section 8 for raw environments).

Comment stripping runs right after fenced code blocks are protected, and before display math / raw environments are protected — so that any `\begin{...}`-like text that happens to land inside a comment is removed before it could ever be mistaken for a real raw environment.

9.1. Line comments — `linecomment: true`

One thing LaTeX users often miss is a plain single-line comment. Setting `linecomment: true` in `mapping.yaml` makes any line starting with `//` **at the very first column, with no leading whitespace at all** disappear entirely. It is off by default.

```
linecomment: true
```

```
First paragraph.

// This line is removed.

Second paragraph.
```

- `%` is off the table since it's both an escape target and common punctuation (percentages); `#` is already used for headings and cross-references; `--` already carries meaning as an en-dash convention (`1--6`) and would collide. `//` was chosen for what's left over.
- **Any indentation at all cancels it.** A line like `  // text` with leading whitespace is left untouched — it's more likely to be text the author actually meant to write.
- **Inside a fenced code block, this never fires, even with the option on.** Fences are protected before this check ever runs, so a genuine `//` comment in a C or JavaScript example starting at column one is safe. The only real risk is a `//` that happens to sit at column one in ordinary prose, which is why the default is off.
- **The following newline is removed along with the line.** That's why a comment can sit between two list items with no blank line and not split the list — which is the whole reason this feature exists.

```
- First item
// A note about this item
- Second item
```

```
\begin{itemize}
\item First item
\item Second item
\end{itemize}
```

Written with blank lines around it instead (`paragraph\n\n// note\n\nparagraph`), those blank lines are untouched, so the paragraph break stays exactly as it normally would.

---

10. Code blocks

10.1. Ordinary code fences

````
```python
def f(x):
    return x + 1
```
````

````
```text
just becomes a verbatim block
```
````

````
```

```raw
\centering
this is assumed to already be LaTeX and passes through untouched.
```
````

- Language tag `raw` / `tex` / `latex` → content passes through **completely unmodified**.
- No language tag, or `text` / `verb` → `\begin{verbatim}...\end{verbatim}`.
- Any other language tag → `\begin{minted}{language}...\end{minted}` (the default `codeblock_template` override it via `mapping.yaml`).
- Inline code `` `code` `` becomes `\texttt{}`, with its contents auto-escaped (backslash included).
- `\`` (backslash + backtick) outputs a literal backtick character. This escape only applies **while scanning for a new code span** — it does *not* apply when searching for the closing delimiter of a span that's already open. That distinction matters: without it, a perfectly normal code span whose content ends in a backslash (e.g. `` `\\` ``, showing the TeX line-break command) would get its closing backtick misread as an escape and the span would break.
- **If text follows right on the line after a closing fence, with no blank line in between, that text always ends up on its own new line** — it doesn't get glued onto the same line as `\end{verbatim}` and the like. Some verbatim-style packages let the catcode changes made by that environment leak into whatever follows `\end{...}` if it's on the same line, so this line break matters for correctness. If the source actually has a blank line (making it a new paragraph), that blank line is kept as well — this rule is only about text that continues immediately after the fence.

````
This is safe even right before a real code delimiter: \`code`
→ This is safe even right before a real code delimiter: `code`

A forced line break is `\\`.  (a normal code span - stays intact)
→ A forced line break is \texttt{\textbackslash{}\textbackslash{}}.
````

10.2. `showexam` — source and result side by side

A block for documents that have to show "write this, get that" — documents like this one (added in 0.2.1). The left box holds the markdown source verbatim; the right box holds that same source **run through the converter a second time**.

````
```showexam
& $ 100% *italic* **bold**
```
````

That single block expands to:

```
\par\medskip\noindent
\begin{minipage}[t]{0.4600\linewidth}\vspace{0pt}
\setlength{\topsep}{0pt}\setlength{\partopsep}{0pt}
\begin{minted}[frame=single,breaklines,breakanywhere,fontsize=\small,%
               xleftmargin=0pt]{markdown}
& $ 100% *italic* **bold**
\end{minted}
\end{minipage}\hfill
\begin{minipage}[t]{0.5000\linewidth}\vspace{0pt}
\setlength{\topsep}{0pt}\setlength{\partopsep}{0pt}
\& \$ 100\% \emph{italic} \textbf{bold}
```
```

```
\end{minipage}
\par\medskip
```

This is not something you can build on the LaTeX side, because it needs the same source written out twice and there is no way to take content that has already been typeset verbatim and feed it back through as markup. So the script holds onto the source and emits it twice: once untouched, once run through the pipeline.

The right box goes through the **exact same pipeline** as the body text. Lists, tables, math, custom environments and cross-references all work normally, and a footnote referenced inside the example resolves against the enclosing document's footnote definitions.

What follows is not a description but something this document actually typeset — set to `style=verbatim` so it compiles with no extra packages.

```
**Bold** and *italic*, plus `code`.

- lists work
- here too
```

Bold and *italic*, plus `code`.

- lists work
- here too

Options on the fence line

Write space-separated `key=value` pairs on the fence line to override the defaults for that one block.

```
```showexam ratio=0.40 style=lstlisting align=c frame=none
```
```

Option	Default	Meaning
`ratio`	`0.46`	Width of the left (source) box, as a fraction of `\linewidth`
`gap`	`0.04`	Space between the boxes. The right box gets `1 - ratio - gap`
`align`	`t`	Vertical alignment of the minipages: `t` / `c` / `b`
`style`	`minted`	How the source box is typeset (see below)
`lang`	`markdown`	Language tag handed to minted
`frame`	`single`	Border around the source box. `none` draws nothing
`layout`	`side`	`side` / `stack` (top and bottom) / `auto` — see "Stacked layout" below
`stack_width`	`36`	Threshold for `auto`: display width of the longest source line (CJK = 2)
`sep`	`rule`	Between source and result in `stack`: `rule` / `space` / `none` / LaTeX code

Wrap a value in double quotes if it needs to contain a space (`lang="text"`). There is also a `listing=` option that replaces the whole listing template for one block, but templates are full of spaces and braces and are awkward to write on a fence line — use `mapping.yaml` for that.

Choosing a `style` — what really decides it is multilingual text

`lstlisting` cannot handle Unicode mixed into the code. Source code itself is usually plain ASCII and gives no trouble, but a single Korean character in a comment is enough to break the `listings` package. For Korean-language code examples, `minted` is effectively the only option — `listings` is built on pdfTeX-era font encodings that were never meant for that.

`style`	Multilingual (Korean etc.)	Wraps long lines	Requires
`minted`	**yes**	yes	the `minted` package (see below)
`lstlisting`	**no**	yes	the `listings` package, nothing more
`verbatim`	yes	**no**	nothing at all

`verbatim` cannot wrap, so a single long line in the source will run straight through the right-hand box and out into the page margin. If you want Unicode without an extra package, `verbatim` with short lines is the only way.

Whether `minted` needs `-shell-escape` depends on when the TeX Live installation was built. **From minted 3.0 on, the call into pygments is handled by a separate Python package called `latexminted`, which ships with TeX Live 2025 onward and runs safely without `-shell-escape`.** Older TeX Live installations, or a `minted` below 3.0, still require it. Check with `kpsewhich latexminted.py`.

Nesting — showing the `showexam` syntax itself

To quote a showexam block as an example, open the outer fence with four or more backticks. The rule is the same as CommonMark's: any fence with fewer backticks is swallowed as content. The first example in this section is written that way.

Conversely, a showexam fence sitting inside an ordinary code fence is **not picked up**. Syntax examples written out in a manual will not be mistaken for real blocks and executed.

Nesting a showexam block inside another one does work, and is rendered recursively up to three levels deep. Deeper than that, the innermost blocks are treated as ordinary code fences.

Changing the layout via `mapping.yaml`

Replacing `template` wholesale lets you reshape the boxes however you like. `tcolorbox`'s `sidebyside` exists for exactly this and fits well.

```yaml
# mapping.yaml - needs \usepackage[most]{tcolorbox} in the preamble
showexam:
  ratio: 0.42
  frame: none      # the tcolorbox already draws a border
  template: |-
    \begin{tcolorbox}[sidebyside,sidebyside align=top,
                      lefthand ratio=%(ratio)s]
    %(left)s
    \tcblower
    %(right)s
    \end{tcolorbox}
```

Leave out `frame: none` and you get **a box inside a box**. The default `template` draws no border of its own, so the listing draws one with `frame=single`; switch to a template that already has a border, like `tcolorbox`, and the two stack up. If you want the inner border back for one particular block, write `frame=single` on that block's fence line.

`sidebyside` and `breakable` **cannot be combined**. Give tcolorbox both and it quietly drops to a stacked top/bottom layout instead — no error, so it is easy to miss. Drop `breakable` to keep the side-by-side arrangement.

The values substituted into the template are:

Placeholder	Contents
`%(left)s`	The source box, already assembled as `verbatim` / `minted` / etc.
`%(right)s`	The converted result
`%(lw)s` / `%(rw)s`	Left / right width factors, as decimal strings like `0.4600`
`%(ratio)s`	The `ratio` value as given — for things like `lefthand ratio`
`%(align)s`	`t` / `c` / `b`
`%(strut)s`	`\vspace{0pt}` when `align=t`, otherwise an empty string

The `styles` entry maps each `style` name to its listing template, so those can be swapped too. Those templates take `%(src)s` (the source), `%(lang)s` and `%(frame)s`. `%(frame)s` expands to an **option fragment including its trailing comma**, like `frame=single,`, and to an empty string when `frame` is `none`.

There is a reason the default template puts `%(strut)s` (i.e. `\vspace{0pt}`) at the head of each minipage and zeroes `\topsep`. A `[t]` minipage aligns on the **first baseline** of its contents, so as soon as the left-hand listing gains a `frame` or any leading vertical space, the two columns visibly drift apart. This idiom switches the alignment reference to the **top edge** of the box instead. It is dropped automatically when `align` is `c` or `b`, where it would do more harm than good.

Stacked layout — `layout=stack` (0.20.1)

When one line is too long for two columns, the source box takes the full text width and the result follows below it, divided by a thin rule (`sep=rule`).

````
```showexam layout=stack
a very long line …
```
````

- `layout=auto` picks stack when the display width of the longest source line (CJK characters count as 2) exceeds `stack_width` (default 36), otherwise side. The real box width depends on paper and font, so this is a rough estimate. `layout=` on the fence line always wins.
- Set the document-wide default with `showexam: {layout: stack}` in the mapping. The stacked template is `stack_template`, using `%(left)s`, `%(right)s` and `%(sep)s`.
- `ratio`, `gap` and `align` mean nothing here and are ignored (with a warning if written on the fence line).
- Stack uses no minipage, so it **can break across pages**. With `lstlisting` the framed box splits; if a bordered box cannot break (for instance minted with `frame=single`), switch to `frame=leftline` or `lines`.

Caveats

- In the side layout the boxes are `minipages`, so they **cannot break across a page**. A long example gets pushed wholesale onto the next page. Switching to `tcolorbox` does not help — `breakable` is incompatible with `sidebyside`. For long examples use `layout=stack` above.
- A `showexam` block **indented inside a list item or blockquote is not supported**. The placeholder is emitted at column zero and breaks out of the list. Use it at the top level only.
- When editing `template`, never put the source box inside a **macro argument** such as `\fbox{...}` or `\parbox{...}` — `verbatim` breaks there. The bodies of `minipage` and `tcolorbox` are environment bodies rather than arguments and are safe; if you want a border, use the `frame=` option of `minted` or `lstlisting`.

11. Lists

```
- **First item**: this sentence is wrapped
  across several lines, and they all get
  joined into one item body.
- Second item
  - Nested item A
  - Nested item B
    - Even deeper nesting

1. Numbered list
2. Next item
1) Parenthesis style also works

- Bullets may contain numbers,
  1. like this,
  2. and the kind may change at every level
     - like this too

{Term} ::   Definition (preferred - braces plus two or more spaces)
{Another term} ::  Another definition

Term :: Definition (the older form, still accepted)
```

- Bullet markers: `-`, `*`, `+`. Numbers can use `1.` or `1)` (the actual number is ignored; LaTeX's `enumerate` renumbers automatically).
- **Item continuation**: a following line with no bullet/number/`::` marker is appended to the previous item's body, joined by a space (or `\\` if the line ended with two trailing spaces).
- **Nesting**: only marker lines indented further than the current level are recursed into as a sub-list. The kind of sub-list is decided by **that line's own marker** — put numbers inside bullets, bullets inside numbers, or a description list inside either, and switch kind at every level if you like.
- Where one level mixes different markers, the **first item** at that level decides the environment. A bullet item landing inside a description list, for instance, falls back to an `\item` with no label.
- Adding `{!options}` on an item's first line applies to the whole list's `\begin{itemize}[options]`. **Only a brace block that starts with an exclamation mark (`!`) is an option** — a `{...}` without the `!` is not an option but part of the item's text, printed literally like any other braces in body text. The `!` is stripped. Extra text after a comma (like `{!1., \firmlist}`) is appended verbatim after the bracket, e.g. `\begin{enumerate}[1.] \firmlist`. This is only recognized on the **first item** of a list.
- **For packages like `enumitem`, where the options themselves are a comma-separated list, wrap the whole bracket portion in double quotes.** Splitting on the first comma (as above) would cut something like `nosep,label={\arabic*},ref=\arabic*` apart at the wrong spot, since the options themselves are full of commas. See below for an example.

```
1. {!"nosep,label={\arabic*},ref=\arabic*",\tightlist} item

\begin{enumerate}[nosep,label={\arabic*},ref=\arabic*] \tightlist
```

Commas **inside** the quotes are never treated as separators — the whole thing goes into the bracket as one piece. Whatever comma comes right after the closing quote is where the raw LaTeX appended afterward begins. Nested braces inside the option block, like `label={\arabic*}`, are matched correctly since braces are counted, not just scanned for the first closing one.

- **Block elements inside an item are not supported.** Leave a blank line in an item body and follow it with a second paragraph or a code block, and the list closes at that point, the block comes out at top level, and a fresh list opens after it. An item body is always a single inline run (see section 22).
- **When a paragraph happens to start with something that looks like a list marker, wrap it in braces to keep it from being read as one.** A sentence like `1. Estimated time is...` gets parsed as a numbered list purely by accident. Wrapping the marker character in braces (punctuation stays outside the braces) prints it literally instead. This check runs only on the **first line** of a paragraph, and **before** the real marker check (`UL_RE`/`OL_RE`) — so there's no risk of the unwrapped result being mistaken for a list afterward. It's unrelated to writing a real list option block (`{!options}`) after an actual marker — that block comes **after** an already-recognized marker, while this escape stands **in place of** the marker at the very start of the line, so the two never collide.

Write	Prints as
`{-}` / `{*}` / `{+}`	`-` / `*` / `+`
`{1}.`	`1.`
`{1})`	`1)`

- **There are two notations for description lists.** Prefer the braced one in new writing.

Notation	Requirement
`{term} :: definition`	line starts with `{..}`; **two or more** spaces/tabs after `::`
`term :: definition`	the three conditions below (older form)

- The braced form is essentially immune to false positives, since prose never accidentally takes that shape, and it lets the term contain spaces or colons — `{C++ std::vector} :: a dynamic array`. Setting `description_syntax: braces` in `mapping.yaml` accepts only the braced form and **prints a warning** on the older one, so migrating an existing manuscript does not silently turn description lists into paragraphs.
- **Three conditions** make the older form a description-list item: (1) at least one space precedes the `::`, (2) the `::` is not part of a longer run of colons such as `:::`, and (3) the term is not empty. So `std::vector`, `12::30` and `:::{name}` are left alone. A `::` inside an inline code span is skipped too, which makes sentences that quote the syntax itself — `` `term :: definition` `` — safe.
- What still cannot be distinguished is a genuinely space-surrounded `::` in prose, as in `he said :: no`. There is no syntactic way to tell those apart — if such sentences are likely in your manuscript, shut the older form off with `description_syntax: braces`.
- **Description lists can also carry a `{option}` block on the first item.** The same `{!option}` syntax used by itemize/enumerate (see above) works after `::` too, and gets pulled out as `\begin{description}[...]`. This is exactly what's needed to give a description list its own Beamer overlay default, as in `\begin{description}[<+-| alert@+>]`. See below for an example.

```
{uncover} ::  {!"<+-| alert@+>"} visible on the given slides.
{only} ::  show on the given slides.
```

```
\begin{description}[<+-| alert@+>]
\item[uncover] visible on the given slides.
\item[only] show on the given slides.
\end{description}
```

The option block is stripped out of that item's **definition** only — the term (`{uncover}`) is untouched. Wrap a comma-containing option in double quotes exactly like itemize/enumerate does (`{!"a,b,c"}`) if the whole thing needs to stay together.

- **When an item genuinely starts with a literal bracket, whitespace between the marker and the bracket decides what it means.** LaTeX treats a `[...]` immediately after `\item` as that item's custom label — handy, but a nuisance when the item body is really meant to start with a literal bracket (blocking that reading with `\item \relax [...]` is a known LaTeX special case). This dialect distinguishes the two by whether the marker and the bracket are written together or with a space. See below for an example.

```
-[def] this bracket becomes the label
- [ordinary text] this bracket is just body text

\item [def] this bracket becomes the label
\item \relax [ordinary text] this bracket is just body text
```

This check runs **only after inline processing, and only if a literal bracket actually survives** — if `- [text](url) starts an item`, the bracket was already consumed by link syntax, so no `\relax` is needed (there's nothing to fix). The same rule applies inside nested lists, and to `enumerate` (numbered lists) as well — `\item[label]` is just as much a custom label there.

---

12. Blockquotes

```
> First line of the quote
> Second line of the quote
```

The contents are recursively re-processed through `render_blocks`. **However, if you just keep writing > lines back to back, they all merge into a single paragraph.** To put a separate block — a list, a heading, etc. — inside a blockquote, you need a blank-line separator just like ordinary text, except inside a blockquote that "blank line" must itself be a bare > line.

```
> First paragraph
>
> - List item A inside the quote
> - List item B inside the quote
```

(Without a line that's just > and nothing else, the second paragraph/list never becomes its own block — even the - list marker gets absorbed as plain text.)

12.1. `quote` versus `quotation`

Which environment wraps the quote depends on **how many paragraphs it has**.

Quote	Environment
one paragraph	`quote`
two or more (a bare > line in the middle)	`quotation`

This follows LaTeX convention: `quote` has no paragraph indentation and suits a short single-paragraph quotation, while `quotation` indents the first line of each paragraph and suits longer, multi-paragraph ones.

```
> The first paragraph.
>
> The second paragraph.
```

```
\begin{quotation}
The first paragraph.

The second paragraph.
\end{quotation}
```

- The decision is made over the range **excluding leading and trailing blank lines**, so a quote that merely begins or ends with a bare > is still a `quote`.
- A genuinely blank line — one without even a > — splits the block into **two separate quotes**. Paragraph breaks within one quote must be written as bare > lines.
- The environment names are configurable in `mapping.yaml`. Setting both to the same value removes the distinction.

```yaml
blockquote_env: quote
blockquote_multi_env: quotation
```

---

13. Horizontal rules and mid-sentence dashes

```
---     ← \noindent\rule{\linewidth}{0.5pt} (default; change via `hr` in mapping.yaml)

***     ← \bigskip (default; change via `asterism` in mapping.yaml)
```

The entire line must consist of 3+ - characters, or 3+ * characters, alone.

A --- in the middle of a sentence (not on its own line — mixed in with other text on the same line, exactly three dashes with no adjacent dashes) is handled differently from the rule above: it's replaced with the default `\textemdash{}` (since manuscripts commonly use this for an em-dash effect). That's a standard LaTeX command, so nothing needs to be defined in `preamble` for it to compile.

```
This uses a mid-sentence dash---for emphasis---like this.
→ This uses a mid-sentence dash\textemdash{}for emphasis\textemdash{}like this.

An en-dash-style -- (two dashes) or ---- (four or more) is left untouched.
```

All three (the rule `hr`, the asterism `asterism`, and the mid-sentence dash `mdash`) can be changed via `mapping.yaml`. None of them live in `DEFAULT_CONFIG` — they're fallback values in the code — so check the relevant section of `sample_mapping.yaml` (see its comments) and set only what you need.

```yaml
hr: '\hrulefill'                        # a lighter-weight rule
asterism: '\fancybreak{* * *}'          # typeset actual asterisks (memoir; needs a preamble
mdash: '\ndhmdash{}'                    # your own macro (needs a preamble definition)
```

---

14. Tables

```
| Name | Age | Note |
|:---|:---:|---:|
| Alice | 20 | left/center/right alignment |
| Bob | 21 | the 2nd row must be a separator (`---`) for this to be recognized as a table |
```

- First row = header, second row = alignment separator (`:---`, `:---:`, `---:`, `---`), everything after is body rows. Converts to `\begin{tblr}{alignment}...\end{tblr}` (booktabs-style `\toprule` / `\midrule` / `\bottomrule`; requires the `tabularray` package).

- The column count is fixed by **the header row's column count**; short body rows are padded with empty cells, and extra cells are dropped.
- **Long cells wrap automatically.** When the table's estimated width looks wider than the line, only the wide columns (roughly 8+ Hangul characters) are switched from `l/c/r` to tabularray's `X[l]/X[c]/X[r]` (with width proportions via `co=`) so that they wrap within their column. The estimate is an approximation that counts a Hangul/CJK character as 2 and anything else as 1; a table at or below the `tablewrap` threshold (default 70, about 35 Hangul characters) is **left completely unchanged.** Any `X` column makes tabularray stretch the table to the full line width, which is why this applies only to tables that look likely to overflow. A table that overflows only because it has many short columns is not helped by this rule (hand-write a raw `tblr`). `tablewrap` is a **mapping key** (not front matter): set e.g. `tablewrap: 60` in `mapping.yaml` or `mappingrule:`, and `0`/`false` turns it off. It applies to pipe tables only; raw `tblr`/`tabular` are never touched.
- **Table attribute line — page-breaking tables (`longtblr`).** A line consisting only of `{…}` placed **immediately after** the last row (no blank line) holds the table's attributes.

```
| Item | Du Fu | Li He |
|:---:|:---:|:---:|
| … | … | … |
{env=long, caption="Two poets compared", label=tbl:cmp}
```

With `env=long` (or `longtblr`) it becomes `\begin{longtblr}[caption={…}, label={…}]{colspec={…},row` the table continues across pages and the header row repeats on each page (`rowhead=0` turns that off). `caption` and `label` are `longtblr`'s own optional arguments, so a caption and a `[#tbl:cmp]` reference work without a float wrapper (`:::{table}`). Without `env`, or with `env=tblr`, it stays a `tblr` as before; `caption`/`label` then have nowhere to go, so a warning is printed and they are ignored (use `:::{table}` if you need a caption). Unknown keys also warn. With a blank line between the table and the `{…}` line, it is just an ordinary paragraph. `longtblr` needs two compile passes to settle page breaks, which `--latexmk` and ksruncltex repeat automatically. The `X`-column auto-wrapping from `tablewrap` above applies as well. If your document spaces tables with `\AddToHook{env/tblr/before}{…}`, add the matching `env/longtblr/before` and `after` hooks too for the same spacing (the default header has no such hooks). - To put a literal vertical bar inside a cell, escape it as `\|`. This is the same rule as ordinary markdown: it is not treated as a separator and a plain `|` is left in its place. - A hand-written raw `tabular`/`tblr` etc. is already protected and passes through under the raw-environment rule from section 8.

14.1. Grid tables — merged cells and multi-line cells

pandoc's grid tables are read too, for when you need merged cells (rows or columns) or multi-line cells. Unlike pipe tables, a cell may contain `|` with no escaping.

```
+---------------------+-----------------------+
| Location            | Temperature 1961-1990 |
|                     | in degree Celsius     |
|                     +-------+-------+-------+
|                     | min   | mean  | max   |
+=====================+=======+=======+=======+
| Antarctica          | -89.2 | N/A   | 19.8  |
+---------------------+-------+-------+-------+
```

Location	Temperature 1961-1990 in degree Celsius		
	min	mean	max
Antarctica	-89.2	N/A	19.8

- **Recognition:** a table whose first line looks like `+----+----+` is a grid table candidate. Blank lines are needed before and after, as for pipe tables.
- **Header and footer:** everything above the first `=` line is the header (several lines are fine); if there is a second `=` line, what follows is the footer (separated with `\midrule`). Without an `=` line the table has no header.
- **Alignment:** set by colons on the header separator (the `=` line) — `+:====+` (left), `+:===:+` (centre), `+====:+` (right). Without a header, the colons on the top line are used. Same rule as pandoc.
- **Rules:** not every `-` line is drawn. As with pipe tables only the three booktabs rules (top, below the header, bottom) are drawn, and only partial separators inside the header (below `Temperature` above) become `\cmidrule`.
- **Cell content:** lines are joined with a space and a blank line becomes a paragraph break (`\par`). Only plain text and inline formatting are allowed in a cell — block elements such as lists and quotes are not interpreted yet.
- **Column widths:** natural width by default (alignment letters only), with the same `tablewrap` as pipe tables turning only overflowing columns into `X`. To get pandoc's behaviour — every column as `X[align,co=width]` in proportion to the dashes of the border — put `{widths=dash}` on the line right after the table.
- **Attribute line:** same as for pipe tables: `{env=long, caption=…, label=…, rowhead=N}`; `rowhead` defaults to the number of header lines. `widths` is for grid tables only.
- **Korean text:** column edges are matched by **display columns** (a Hangul syllable is two), not by character index, so a table lined up by eye in a monospaced editor reads correctly. If that does not parse, a table that counts Hangul as one column is read too.
- **Errors:** a table whose corners do not close, or whose `|` do not line up with the lines above and below, is reported by `--lint` as an error (with the line number); converted, it becomes an ordinary paragraph (with a warning).

---

15. Figures

```
![Caption text](path/to/image.png){width=0.75\linewidth, label=fig:foo, place=htbp}

![](path/to/image.png)   ← no caption/label: outputs only \includegraphics

![""](path/to/image.png) ← wrapped in figure, but with no \caption
```

- Inside `{...}`: comma-separated `key=value` pairs or bare keys. `label`, `place` and `align` are handled specially — they become `\label{}`, the `\begin{figure}[place]` placement option, and the alignment command placed before the image — and **everything else becomes an `\includegraphics[...]` option**. `place` defaults to `htbp` and `align` to `\centering`.
- The `align` value is emitted raw, so write the LaTeX command you want (`align="\raggedleft"`). An explicit empty value (`align=""`) omits the alignment command entirely. This matches the `:::{figure}` environment of 17.1.
- Remember that everything outside those three goes to `\includegraphics`. A key graphicx does not know breaks the build with `Package keyval Error: ... undefined`.
- The alt text and `label` select one of three outputs.

Manuscript	Output
`![](x.jpg)`	`\includegraphics` alone, not wrapped in a float
`![text](x.jpg)`	a `figure` environment with `\caption{text}`
`![""](x.jpg)`	a `figure` environment with no `\caption`

- The third form is how you get a **captionless figure float**. It applies only when the alt text is a single empty pair of quotes (`""`, `''`, `""`, ‘’); anything inside the quotes makes it an ordinary caption.
- **Quotes are not caption delimiters.** `!["text"](x.jpg)` yields `\caption{``text''}`, quotes included, after the smart-quote conversion of section 17. Only a completely empty pair is special-cased.
- **A link can be nested one level into the alt-text slot.** Writing `![[label](url)](x.jpg)` makes the caption itself a link (example below). Only one level of bracket nesting is recognized, so anything deeper is not. A `%` inside the `url` is escaped automatically (see section 16). This only works in the alt-text slot — a `caption=` attribute value does not get `[label](url)` reprocessed, since an attribute value is raw text for that environment; write `\href{...}{...}` by hand there if a link is needed.

```
![[Pi](https://en.wikipedia.org/wiki/Pi_%28letter%29)](pi.png)
```

```
\begin{figure}[htbp]
\centering
\includegraphics{pi.png}
\caption{\href{https://en.wikipedia.org/wiki/Pi_\%28letter\%29}{Pi}}
\end{figure}
```

- **A `caption=` attribute and the alt text mean different things.** Alt text floats the image in a `figure`; `caption=` leaves it where it stands and only attaches a caption (a “dry” caption). See the table below. `\captionof` comes from `capt-of` (or `caption`), and the numbering and `\label` join the same sequence as floated figures. Loading it is the author’s job.
- **`inline`**, as a bare key, emits `\inlinegraphics` — for small images set into running text, which requires the `inlinegraphicx` package (again the author’s job). The `inlinespec=` value is passed to that command’s bracket argument **verbatim**; without it the other attributes go there. The command name is configurable as `inline_graphics_cmd` in `mapping.yaml`.
- **`wrap`**, given as a bare key, replaces the floating `figure` with a layout in which the body text flows around the image (requires the `wrapstuff` package; loading it is the author’s job).

```
![](x.png){wrap, wrapspec="type=figure,width=5cm", caption="test",
           label="fig:wrap", width=3cm, align="\centering"}
The paragraph continues here, with no blank line after the image.
```

```
\begin{wrapstuff}[type=figure,width=5cm]
\centering
\includegraphics[width=3cm]{x.png}
\caption{test}
\label{fig:wrap}
\end{wrapstuff} The paragraph continues here, ...
```

- The `wrapspec=` value is passed to the environment’s bracket argument **verbatim**; this script does not interpret it, so whatever the environment accepts is up to you. Quote it — the value contains commas.

- The layout requires the **environment to sit at the start of a paragraph**. Writing body text on the line right after the image, with no blank line, achieves that: image markup is processed inline and therefore stays in the same paragraph as the text that follows.
- `place` is unused with `wrap` (nothing floats). The environment name is configurable as `wrap_env` in `mapping.yaml`.
- **`href=`** puts a link on the image itself (example below). Only the `\includegraphics` is wrapped, not the caption or label — the same inside a `figure` and under `wrap`. A `%`, `#` or `&` in the URL is handled by hyperref, so write it as it is. `\href` comes from hyperref, so loading it is the author's job (the default header is a good place).
- Among the options forwarded to `\includegraphics`, `center` comes not from graphicx but from `adjustbox` (`\usepackage[export]{adjustbox}`) — again, the author's job.
- A `label` with no caption leaves `\label` unable to pick up a number, so `\ref` yields whatever the counter happened to hold; a warning is printed. To reference an image without floating it, add `caption=` — `\captionof` takes care of the numbering.
- The caption text is recursively run back through `process_inline`, so bold, italics, quote conversion, etc. all work correctly inside captions too.

Captions and floating:

Whether the image floats is decided by the alt-text slot alone. With empty brackets, nothing written in `{}` will wrap it in a `figure`.

Manuscript	Result
`![caption](x.png)`	a `figure` environment with `\caption`
`![""](x.png)`	a `figure` environment, no caption
`![](x.png){caption="caption"}`	left in place, with `\captionof{figure}`
`![](x.png)`	`\includegraphics` alone

```
![](myfig.png){caption="dry caption", label=fig:d}
```

```
\includegraphics{myfig.png}
\captionof{figure}{dry caption}
\label{fig:d}
```

A small image in running text:

```
![](smallfig.png){inline, inlinespec="scale=1.5,strut={()}"}
```

```
\inlinegraphics[scale=1.5,strut={()}]{smallfig.png}
```

Linking an image:

```
![](image.png){scale=.3, href="https://ctan.org"}
```

```
\href{https://ctan.org}{\includegraphics[scale=.3]{image.png}}
```

15.1. Fetching and converting images

Three separate concerns, three separate switches.

Concern	Switch	Default
Network access	`--fetch-images`	off
Format conversion	none (done when Pillow is available)	on
Animation	`--animation[=N]`	off

Only downloading is opt-in, and the reason is **security** rather than reproducibility: converting a `.md` someone sent you should not by itself issue requests to arbitrary addresses. Conversion, by contrast, merely writes one more file, and it would be odd to make someone with a local `.webp` — no URL anywhere — turn on a download option.

Images given by URL

```
python3 ndhmd4tex.py circular.md --pdfxe --fetch-images
```

One extra flag on the usual command. Files land in `_images/` beside the manuscript, named by **a hash of the URL plus the real extension**. The last path segment is deliberately not used: names like `330px-...` collide across sources, and percent-encoding and Unicode make poor filenames.

- Anything already downloaded is reused, so **builds after the first work offline**.
- **A cached file in `_images/` is used even without `--fetch-images`.** What the option guards is the outgoing request, not the use of a file already in hand, so once something has been fetched the usual command builds it.
- The extension comes from `Content-Type` first; a URL path may carry none, or the wrong one, and a 404 page may arrive with status 200.
- With neither `--fetch-images` nor a cached copy, that one image is left as written and reported; the rest of the conversion continues.

Paths are always written with **forward slashes**. A LaTeX source uses `/` regardless of the operating system; a stray backslash from Windows would make `_images\dn.jpg` read as an undefined `\d` command and break the build.

Format conversion

LaTeX accepts `.pdf`, `.png`, `.jpg`, `.eps` and `.bmp` directly. (That `.bmp` works is a measured fact — contrary to common belief, xelatex takes it.)

Format	Handling
`.tif .tiff .webp .gif`	converted to PNG with Pillow
`.svg .ai .psd`	kept as-is, with a note

The converted file is written **beside the original**: `photo.webp` becomes `photo-webp.png`. Keeping the original extension in the name stops `p.webp` and `p.tiff` from both becoming `p.png` and overwriting each other. Originals are never deleted, leaving room to reconvert later with a better tool — and a way back if a link dies.

Pillow is an optional dependency. Without it the tool works normally and simply explains what to install when it meets a format that needs converting.

`.svg` is left alone because rasterizing a vector is a loss, and converting to PDF needs inkscape or rsvg-convert, neither of which is guaranteed to exist. You get the file; what to do with it — convert it, use `\usepackage{svg}` with `-shell-escape`, or pick a different image — is your call.

Animation (not a selling point)

Almost no viewer plays it. The `animate` package relies on JavaScript, so it moves in Adobe Acrobat and PDF-XChange and shows only the first frame in Preview, Skim, Okular and browser viewers. Use it knowing that.

```
python3 ndhmd4tex.py doc.md --animation      # first 40 frames
python3 ndhmd4tex.py doc.md --animation=12   # first 12
python3 ndhmd4tex.py doc.md --animation=all  # all of them, on your head be it
```

- Frames go to `name-gif-frames/` as `0.png` onward — one file per frame, so three 30-frame GIFs leave 90 files behind. Hence the default cap of 40.
- **Per-frame delays cannot be preserved.** A GIF may time each frame differently; `\animategraphics` takes a single fps, configurable as `animation_fps` in `mapping.yaml` (default 10).
- Frames are taken from the front. Sampling evenly would keep the loop intact but blur the timing a second time, after the fps has already blurred it once.
- Without `--animation`, an animated GIF contributes its first frame, and says so.
- **Already-extracted frames are reused.** If enough of them already exist, nothing is re-encoded. Raising the cap (say to `--animation=all`) and finding a shortfall triggers a fresh extraction from scratch; lowering it just reuses what's there (the leftover frame files in that folder are harmless). If the source GIF changed but happens to have the same frame count as before, this won't notice — delete the `*-frames/` folder and rerun in that case.

15.6. Reusing a reference multiple times — tagged references

When the same figure (or link) needs to be pointed to more than once, MultiMarkdown-style tagged references let you define the path/URL once and reuse it by a short label.

```
![Figure caption][myfig]
...
[myfig]: figurename.png width=3cm,respectratio

For more, see [][Markdown].
...
[Markdown]: https://en.wikipedia.org/wiki/MultiMarkdown
```

```
\begin{figure}[htbp]
\centering
\includegraphics[width=3cm,respectratio]{figurename.png}
\caption{Figure caption}
\end{figure}
...
For more, see \url{https://en.wikipedia.org/wiki/MultiMarkdown}.
```

- **A definition is a single `[label]: value` line** (multi-line continuation isn't supported — an image path plus attributes, or a URL, almost never needs more than one line in practice. Writing more lines just leaves the second one onward as ordinary body text — it doesn't vanish quietly, so a mistake stays visible). It can go anywhere in the document (before or after where it's referenced) and disappears from the rendered body (the same treatment as a footnote definition).
- **Per-use `label`/attributes go in a `{…}` after the reference.** Attributes on the definition line are the image's shared defaults (path, `width`, …); a `{…}` placed right after the `]` is the value for this figure instance only — the same rule as `![caption](x.png){…}` (with a space in between it is body text, not attributes).

```
![first use][myfig]{label=fig:a}
![elsewhere][myfig]{label=fig:b,width=60pt}

[myfig]: mmd.png width=100pt
```

For the same key the use site wins. An empty value (`{label=}`) removes that attribute from the definition (except `align`, where an empty value already means "no alignment command" and simply overrides). A `\label` must be unique per figure, so a `label=` on the definition line is only safe if the definition is used **once** — used two or more times without a per-use `label`, it would duplicate the

`\label` (LaTeX's "multiply defined"), so a warning is printed. The reference's name (`myfig`) is just the key that finds the definition and is unrelated to LaTeX labels. Link references (`[text][label]`) have no attributes, so this does not apply. - **Figure references and link references share the same label pool.** A single `[label]: value` can serve as either an image or a link — which one it becomes is decided by how it's referenced (`![alt][label]` vs. `[text][label]`). A figure definition's value is "path plus (if given) attributes" (`figurename.png width=3cm,respectratio`, the same syntax as 15.1's `{width=...}` attributes, just without the braces); a link definition's value is a bare URL. - **The bare single-bracket shortcut (`[Markdown]` alone) is deliberately not supported.** There's no way to tell it apart from a bracket used for emphasis or an aside in ordinary prose, which would go against this dialect's stance of only recognizing explicit syntax. Instead, only the **double-bracket** form (a `][` sequence) is accepted: `[][label]` (empty display text) or `[text][label]`. This sequence essentially never occurs in ordinary prose, so there's no ambiguity, and it doesn't collide with `[[term]]` (index), `[^id]` (footnote), or `[#id]` (cross-reference) — each starts with a different character. - `[][label]` (empty display text) follows exactly the same rule as `[](url)` — for a link that's `\url{...}`, for a figure that's an insert with no caption. Not having to type the URL or filename twice is the whole point of this syntax. - Referencing an undefined label (a typo, say) doesn't halt with an error — it prints a warning and leaves the original text untouched. Write `![][no-such-label]` and that's exactly what shows up in the output — a compromise that doesn't block compilation while still making the mistake visible.

---

16. Links / footnotes / cross-references / index entries

16.1. A label anywhere — `{#id}`

`# Title {label=...}` (section 3) and `![caption](x){label=...}` (section 15) are attribute syntax specific to headings and figures. But sometimes a label is needed somewhere that has no attribute syntax of its own — a list item, a footnote. `{#id}` fills that gap: it turns into `\label{id}` **anywhere inline**.

```
- First item{#item:a}
- Second item

Footnote text can carry a label too{#fn:a}.

[^f1]: Even inside a footnote definition{#fn:b}.

Reference: [#item:a]
```

```
\begin{itemize}
\item First item\label{item:a}
\item Second item
\end{itemize}

Footnote text can carry a label too\label{fn:a}.
...
Reference: \ref{item:a}
```

- Label names accept only letters, digits, `:`, `_`, `-` and `.`. Outside that shape (a space, say) it is not converted and `{#...}` is left as written — with no error, so if a label doesn't take, check the name for a disallowed character first.
- Inside a code span (`` `{#not:a}` ``) it is left untouched.

- Headings and figures already have dedicated syntax, so keep using `{label=...}` or `{#...}` (as an attribute, section 3) there. This section's `{#id}` looks similar but is a separate mechanism that operates in a different place.
- **Placed right after display math, it becomes a dead label.** `\label` picks up whatever counter was most recently updated at that point, and that state is gone the moment `align`/`equation` is left, so `\ref` comes back empty. To reference an equation number, label it **inside the environment** (write `\label{...}` there directly, as raw LaTeX). This section's `{#id}` is for places where the position itself is the thing being referenced — prose, list items, tables — not a substitute for labeling display math.

```
  \begin{align}
  a &= b + c \label{eq:good}   ← works here
  \end{align}
  {#eq:bad}                    ← dead label here
```

```
[link text](https://example.com)
[link text](https://example.com "the title is parsed but discarded")
[](https://example.com)                ← an empty label becomes \url{}
[Pi](https://en.wikipedia.org/wiki/Pi_%28letter%29)  ← a % in the url is escaped automatical
[](#sec:intro)                         ← a url starting with # is a label link instead
[see it here](#sec:intro)              ← \hyperref[sec:intro]{see it here}
```

When the parenthesized part starts with `#`, it's treated not as an external URL but as a reference to a label somewhere in the document. A real URL never starts with `#` — it starts with `http://`, `https://`, `mailto:`, a relative filename, and so on — so there's no ambiguity. `\hyperref[label]{...}` is used rather than `\ref` because the two mean different things: `\ref` only produces a number, while what this syntax wants is a clickable link to that spot. An empty label (`[](#id)`) uses the label name itself as the link text. Loading `hyperref` is the author's job. This is unrelated to `[#label]` (the single-bracket cross-reference syntax, covered below) — the two look similar but never collide.

```
A footnote reference[^note1] in the body text.

[^note1]: The footnote content goes here.  ← id: Latin letters, digits, _, -, Hangul only
    Starting from the second line, indent with 4 spaces
    (or a tab) to continue it.

You can also write one inline[^like this, right here].
Pandoc's notation^[means the same thing] works too.
```

```
See [#sec:intro].          → \ref{sec:intro}
See page [page:sec:intro]. → \pageref{sec:intro}
Cite [@smith2020].         → \cite{smith2020}
[@t:smith2020] argued so.  → \citet{smith2020}   (natbib/biblatex)
It is supported [@p:smith2020]. → \citep{smith2020}
[@a:smith2020] ([@y:smith2020]) → \citeauthor{} (\citeyear{})
[title:sec:intro].         → \titleref{sec:intro}
[name:sec:intro].          → \nameref{sec:intro}
[E:eq:euler].              → \eqref{eq:euler}      (amsmath)
[F:fig:sample].            → \figurename~\ref{fig:sample}   → Figure 1
[T:tab:sample].            → \tablename~\ref{tab:sample}    → Table 1
[C:sec:intro].             → \Cref{sec:intro}
[C:sec:a,sec:b].           → \Cref{sec:a,sec:b}   (cleveref's comma-separated multi-label fo

[[Term]]                       → Term\index{Term}
[[display text|index term]]    → display text\index{index term}
```

`[[|index term only]]` → `\index{index term only}`

- **A % inside the `url` of `[label](url)` is escaped to \% automatically.** `\href` reads its first argument through hyperref's verbatim-scanning trick, and that trick only works when the argument is read straight from the top-level token stream. Once the link is wrapped inside another macro argument — a `\caption{...}` is exactly this, which is what the `caption=` attribute of section 15 produces — the argument goes through ordinary macro expansion instead, and everything in the `url` from the first % onward is swallowed as a LaTeX comment, along with the rest of that line. The actual link target in the final PDF is unaffected: hyperref restores `\%` back to a plain %. An empty label (`\url{}`) never had this problem and is left untouched.
- **A `\href{...%...}{...}` written directly in the body does not get this protection.** If, say, a `caption="..."` attribute contains raw LaTeX with a hand-written `\href`, the % inside it is raw text the script does not touch, so it must be escaped as `\%` by hand.
- **If the same link (or figure) needs to be used more than once, a `[text][label]` reference works too** — write the URL or filename once and reuse it by label. See section 15.6 for details.
- **The index command can be replaced wholesale via `index_template`.** TeX's `\index` reads its argument in a way that makes it very awkward to redefine, and sometimes you want a macro of your own instead. `%(text)s` receives the display text and `%(term)s` the index term. Because the template also decides *where* the display text goes, the case with no display text (`[[|index term only]]`) simply leaves `%(text)s` empty — one template covers all three forms. An unknown placeholder prints a warning and falls back to the built-in `\index` form. See the example below.
- **The index term and display text are emitted raw** — no escaping, no inline formatting. makeindex treats `!` (subentry), `@` (sort key) and `|` (page-format) specially inside `\index`, and touching them would make that syntax unusable. The flip side is that a % or `&` in an index term has to be escaped by hand, and markdown formatting in the display text (as in `[[**bold**|term]]`) is not converted.
- **A standalone definition's identifier may contain only Latin letters, digits, underscore, hyphen, and Hangul syllables** (`[A-Za-z0-9_\-가-힣]+`). `[^note1]`, `[^ndh-2024]`, `[^fn_3]`, and `[^주석1]` are fine; `[^two words]` (with a space) is not. Leaving out spaces is the key part — `[^...]` doubles as both "a reference" and "an inline footnote written right there" (see below), and the only thing that tells them apart is whether the content matches an already-defined identifier. A space makes a short reference and a multi-word inline footnote indistinguishable; spaceless Hangul carries none of that ambiguity, since it behaves exactly like an ASCII identifier does.
- Breaking that rule **fails silently and twice over.** The definition line is not collected, so it stays in the body — where the inline-footnote rule turns it into a `\footnote` of its own — and the reference, finding no definition, typesets the identifier itself as a footnote. A line that looks like a definition but whose identifier breaks the rule therefore prints a warning; if your footnote numbering has grown unexpectedly, check for it first.
- **Footnotes come in a reference form and an inline form.** If what is inside `[^...]` matches the identifier of a collected footnote definition, it is a reference to that definition; otherwise the content itself is taken as the footnote text. Definition identifiers are restricted to `[A-Za-z0-9_\-가-힣]+`, so content containing spaces or other non-permitted characters can never be a reference — which is why the two forms cannot be confused.
- Pandoc's `^[footnote text]` notation is accepted as well. It cannot be mistaken for a reference, so reach for it when you want the intent to be unambiguous in new writing. It does not clash with `^^dotemph^^` (the brackets tell them apart).

- Inline formatting works inside footnote text: `[^**bold** and *italic*]` becomes `\footnote{\textbf{bol and \emph{italic}}}`. Brackets may nest **one level deep**, so `[^see [#sec:a] for this]` works. Two levels of nesting, or a line break, do not.
- **A `^[` immediately following dot emphasis (`^^...^^`) is not read as a footnote.** Without that rule the second of the two closing carets binds to the bracket and swallows what follows — `^^Lebesgue integral^^[[|Lebesgue measure]]` lost its whole index entry. The exclusion makes every combination of dot emphasis followed directly by an index entry or cross-reference (`^^emphasis^^[#sec:a]`) work. To put a genuine inline footnote after dot emphasis, leave a space: `^^emphasis^^ ^[note]`.
- **Careful**: a typo in a reference identifier now typesets silently as a footnote containing the typo — `[^ndh2024x]` becomes a footnote reading "ndh2024x". If that trade is unwelcome, set `inline_footnote: false` in `mapping.yaml` and undefined references go back to printing a warning and being left untouched.
- Footnote definitions can appear **anywhere in the document** and don't need to come after their reference (they're all collected in a pre-processing pass). Referencing an undefined `[^id]` prints a warning to stderr and leaves the reference text unchanged.
- `E:` is amsmath's `\eqref`. The prefix is deliberately not named `eq:`: a prefix is **stripped** from the label, so `eq:` would turn `[eq:c]` — the obvious way to reference a conventionally named `\label{eq:c}` — into `\eqref{c}`. Writing `[eq:eq:c]` to work around that serves nobody, hence the `C:`/`F:`/`T:`-style spelling.
- `F:` and `T:` imitate memoir's `\fref`/`\tref`, sparing you from typing "Figure" or "Table" at every reference. `\figurename`/`\tablename` are defined by every class and swapped for the right word by babel and kotex, which makes them less work than `cleveref` (`C:`) in a Korean document. The ~ keeps the name and the number on the same line.
- `@t:` (`\citet`), `@p:` (`\citep`), `@a:` (`\citeauthor`) and `@y:` (`\citeyear`) are natbib/biblatex commands. In book work the distinction between pulling an author into the sentence (`\citet`) and putting the reference in parentheses (`\citep`) is essentially mandatory, hence their presence in the defaults. A prefix only fires when it is used, so documents that load neither package pay nothing.
- **Citations that take arguments use `{rawopt=…}`.** The value of a `{rawopt="…"}` placed right after the closing `]` is inserted **verbatim** between the command and `{key}`.

```
[@Allen04]{rawopt="[p.~50]"}          →  \cite[p.~50]{Allen04}
[@Allen04]{rawopt="[see][p.~23]"}     →  \cite[see][p.~23]{Allen04}
[@p:Allen04]{rawopt="[cf.][ch.~2]"}   →  \citep[cf.][ch.~2]{Allen04}
```

The value may be quoted (a `]` or `}` inside the quotes is fine) or bare (`{rawopt=[p.~7]}`). A value that does not start with `[` is wrapped in `[…]` automatically (`{rawopt="p.~3"}` → `\cite[p.~3]{key}`). A value that does start with `[` is inserted as is, so you must **balance the brackets yourself** — leaving out the closing bracket as in `"[see][p.~23"` passes `\cite[see][p.~23{Allen04}` through and breaks the compile. It works with every prefix (`@t:`, `@p:`, …); on a custom prefix whose template has no `{%s}` it warns and is ignored. A brace group that does not start with `{rawopt=` stays ordinary body text. Writing `\citep[p.~33]{key}` directly in the body also still passes through under the macro rule of section 7. - The `[prefixlabel]` cross-reference/citation syntax pulls its prefix → LaTeX-template mapping from `cfg["refcite"]` (the thirteen prefixes above are the defaults), and **this mapping can be freely overridden via `mapping.yaml`** — for example, if a project uses its own `\hbref` macro instead of `\nameref`, override just the `name:` entry as shown below; the rest of the defaults stay intact (see the deep-merge behavior in section 19). You can also add entirely new prefixes this way (e.g. `"eq:": '\eqref{%s}'`). - `title:` and `name:` both end up meaning roughly "reference the title," but book editors often want them under separate macro names by convention,

so both are provided by default. `name:` uses the standard `\nameref` from `hyperref`. `title:` assumes a project-defined macro (`\titleref`) — same pattern as `\dotemph`, which also needs to be defined in `preamble`. - `C:` (cleveref's `\Cref`) requires `\usepackage{cleveref}` in `preamble`. To reference several labels at once, chain them with commas — `[C:a,b,c]` — cleveref handles that natively. A "range" reference like `\crefrange{a}{b}`, which takes two separate arguments, is *not* supported through this bracket syntax — thanks to the raw-macro passthrough rule in section 7, you can just write `\crefrange{sec:a}{sec:b}` directly in the body and it passes through unchanged, so there was no need to add new bracket sugar for it. - Keep in mind that a short prefix like `C:` could accidentally match unrelated bracketed text that happens to start the same way (e.g. a Windows path like `[C:\Users\...]`). If that becomes an issue, swap it for a safer prefix (e.g. `"cref:"`) via `mapping.yaml`, same as the example just below.

```yaml
# mapping.yaml - swap only the name: prefix for a project-specific macro
refcite:
  "name:": '\hbref{%s}'
```

Replacing the index command:

```yaml
# swap the command name only
index_template: '%(text)s\myidx{%(term)s}'

# a macro taking display text and index term as two arguments
index_template: '\myidx{%(text)s}{%(term)s}'
```

16.2. Kinds of footnotes — `notes` and the trailing tag

By default every note is `\footnote{…}`. When you need **several kinds** — memoir's `\sidefootnote`, `\pagenote`, `\footnoteA`/`\footnoteB`, as in a critical edition — register them under `notes` in the mapping and pick one with a `{name}` tag right after the note.

```yaml
notes:                          # added to the built-ins (default, side, page, sidepar, sideba
  A: '\footnoteA%(opt)s{%(text)s}'
  B: '\footnoteB%(opt)s{%(text)s}'
  default: '\footnoteA%(opt)s{%(text)s}'    # to change the kind of an untagged note
```

```
A variant reading.^[Witness 1 has 仁.]{A} And …[^k1]{B}
```

Tag	Meaning
`{A}`	a name registered in `notes`; accepted after both `^[…]` and `[^id]`
`{A, opt=3}`	explicit number: `\footnoteA[3]{…}` (`opt="[3]"` also works)
`{sym}`	`\fnsymbol` symbol note (*, †, ‡…), numbered 1, 2, 3… in document order
`{sym=3}`	explicit symbol number (automatic numbering continues from 4); `{sym=1}` restarts
`{A, sym}`	combined with a kind, on the `\footnoteA` family

- Templates may use `%(text)s` (the body) and `%(opt)s` (`[3]` or empty). The old single `%s` form still works.
- **`sym` does not touch the ordinary counter.** It is emitted as `{\renewcommand{\thefootnote}{\fnsymbol{fo` `\footnote[N]` with an explicit number does not step the counter, so no save/restore code is needed. `\fnsymbol` has nine symbols, so the tenth warns. Only the `\footnote` family (including `\footnoteA`) is guaranteed.

- An unregistered name (`{sidee}`) is not read as a tag: it **warns and stays in the text** (LaTeX would silently swallow it). The warning comes at conversion time — `--lint` does not read the mapping, so it cannot know what is registered. Anything containing a backslash (`{\bfseries x}`) and `{}` are not tags and are left alone without a warning, as before.
- `\footnoteA` needs `\newfootnoteseries{A}` in the preamble; `\pagenote` needs `\makepagenote` and `\printpagenotes`. Put them in `preamble:`.
- `\verb|…|` (and `\verb*|…|`) spans pass through untouched **everywhere in the text** (since 0.19.3); `\verbfootnote` (`{verb}`) is the main use.

---

17. Custom environments (`:::`)

```
:::{theorem}
The theorem body. Requires `\newtheorem{theorem}{Theorem}[section]` to be
defined in the `preamble`.
:::

:::{quote}
A pre-defined quote environment.
:::

:::{center}
Centered content.
:::

:::{tcolorbox title="Note" colback=yellow!10}
Custom boxes are also possible, via mapping.yaml or attrs.
:::

:::{adjustwidth raw={1cm}{1cm}}
Body text with the left/right margin widened by 1cm each. Use this for an
environment like adjustwidth that takes unnamed (positional) arguments.
:::
```

- **The syntax requires `{` to immediately follow `:::` with no space**, as in `:::{name attrs}`. If there's a space, as in `::: {name}`, it will not match and the text is treated as plain prose (a currently known limitation).
- **A bracket argument glued directly onto the name, as in `:::{name}[arg]`, is not supported.** Use the `title=`/`raw=` approach below if you need an argument.
- The closing line is a bare `:::` on its own.
- `quote`, `center`, `table`, `figure`, `thebibliography`, `hverse` and `verse` have built-in mappings; every other name produces `\begin{name}…\end{name}` (that environment must actually be defined in your LaTeX setup — whether via `\newtheorem` or a package).
- **Attributes given to an unregistered environment are discarded**, because the `default` template only uses `%(name)s` and `%(raw)s`. A warning is printed when this happens, so that writing `:::{mybox title="Note"}` and wondering where `title` went is not a silent mystery.
- Attributes are parsed by `parse_attrs` as `key=value` or `key="quoted value"`, and can be referenced as `%(key)s` inside the `environments.<name>.begin/end` templates in `mapping.yaml`. **Attribute values pass straight into the template raw, with no escaping or inline formatting applied** — if a value contains LaTeX special characters (`&`, `%`, `_`, etc.), escape them yourself.

- **To put a literal quote inside a quoted value, double it** (the same convention as CSV/SQL string literals). `title="A --- ""B"""` gives a value of exactly `A --- "B"`. A backslash (`\"`) doesn't work — this parser has no concept of escaping at all, so `\"` still ends the value at that very next `"`, backslash or no. For an attribute like `title=` that does go through inline processing, the `"..."` recovered this way still gets converted into proper smart quotes (“...”).
- **Separate attributes with commas.** Figures in section 15 (`{width=..., label=...}`) and heading attributes in section 4 (`{unnumbered, label=...}`) are comma-separated, so this keeps the notation uniform. The parser scans for `key=value` fragments, so space separation still works, but pick one and stay with it within a document. In an unquoted value the scan stops before whitespace and before a comma, so a separator comma never leaks into the value. If the value itself needs a comma, quote it (`raw="{1cm},{2cm}"`).
- For `\begin{theorem}[Pythagorean theorem]`-style **named bracket arguments**, register the environment in `mapping.yaml` and fill the value in via an attribute (see the example below). If the attribute is omitted entirely, the template's `%(title)s` reference raises a `KeyError`, which is caught: a warning is printed and it falls back to the plain `\begin{theorem}...\end{theorem}` form without brackets.
- An environment like `adjustwidth` that takes **several unnamed (positional) arguments** doesn't fit the `key=value` attribute syntax at all, since the arguments have no names. For that case, the attribute name `raw` is treated as reserved: its value is inserted, unescaped, immediately after `\begin{name}` (the default environment template is literally `\begin{%(name)s}%(raw)s`, so this works with no `mapping.yaml` registration at all — see the example below). Environments that don't use `raw` are completely unaffected.
- If a `raw` value needs to contain a space (e.g. `\hspace{1cm} {1cm}`), quote it: `raw="\hspace{1cm} {1cm}"` — an unquoted value is only read up to the first space.
- `:::` blocks can be nested (tracked via a depth counter), and their contents are recursively processed through `render_blocks`, so you can freely put lists, headings, tables, etc. inside them.
- **If text follows right on the line after the closing `:::`, with no blank line in between, that text does not get typeset as a new paragraph.** Internally a block boundary still occurs there (needed so the contents can be processed recursively), but the absence of a blank line in the source is itself the author's signal that “this isn't a new paragraph” — so it's kept on its own line without the indentation a new paragraph would get. Add an actual blank line and it becomes a real new paragraph, as expected.

```
:::{quote}
A line of text
:::
continuing right into the next sentence like this.
```

```
\begin{quote}
A line of text
\end{quote}
continuing right into the next sentence like this.
```

- **Don't put a `:::` block inside a list item.** A multi-line construct with blank lines inside it — a `:::` block or a code fence — breaks the block at that point, and every list item that follows falls out of the list and gets merged into a single plain paragraph (list-item continuation in section 11 only works within “one block with no blank lines”). Keep code-fence examples and `:::` blocks as their own standalone blocks outside of lists.
- **A single blank-line-free code fence can go inside a list item, but only at the top level.** A fence is only recognized as a fence when its absolute indentation (counted from the very start of the document) is 0–3 spaces (the CommonMark rule) — a top-level list item's indentation (2–3

spaces) falls within that range, but nesting one level deeper necessarily pushes it past 4 spaces, so the fence itself goes unrecognized and the backticks end up as literal text. For code inside a deeply nested list, pull the fence out as its own standalone block outside the list, as in the item above.

Named bracket argument example (`title=` registered in `mapping.yaml`):

```yaml
# mapping.yaml
environments:
  theorem:
    begin: '\begin{theorem}[%(title)s]'
    end: '\end{theorem}'
```

Writing

```
:::{theorem title="Pythagorean theorem"}
...
:::
```

converts to

```
\begin{theorem}[Pythagorean theorem]
...
\end{theorem}
```

Positional argument example (`raw=`, no registration needed):
Writing

```
:::{adjustwidth raw={1cm}{1cm}}
...
:::
```

converts to

```
\begin{adjustwidth}{1cm}{1cm}
...
\end{adjustwidth}
```

17.1. Floats — `table` and `figure`

`table` and `figure` are built in, so they work without registering anything in `mapping.yaml`.

```
:::{table place="htbp", caption="A table drawn with T/B/L/R helpers", label="tab:1"}
\begin{tblr}{colspec={lcr}}
  Head A & Head B & Head C \\
  Data 1 & Data 2 & Data 3 \\
\end{tblr}
:::
```

```
\begin{table}[htbp]
\centering
\caption{A table drawn with T/B/L/R helpers}
\label{tab:1}
\begin{tblr}{colspec={lcr}}
  Head A & Head B & Head C \
  Data 1 & Data 2 & Data 3 \
\end{tblr}
\end{table}
```

Four attributes are recognized.

Attribute	Default	Notes
`place`	`htbp`	The placement option of `\begin{table}[...]`
`align`	`\centering`	Passed through raw, so write a LaTeX command
`caption`	(none)	Without it the `\caption` line is omitted entirely
`label`	(none)	Without it the `\label` line is omitted entirely

`figure` takes the same attributes but puts **caption and label below the content**, following the usual convention of captions above tables and below figures. Use it to wrap something that section 15's `![](...)` syntax cannot express, such as a hand-written TikZ picture.

To switch a default off, give an explicit empty value: `:::{table align=""}` drops the `\centering` line.

The body is processed recursively, so it can hold a markdown table from section 14 just as happily as the hand-written raw `tblr` above (which is protected by the raw-environment rule from section 8).

17.2. `thebibliography` — a reference list written by hand

If you keep a `.bib` file and run biber or bibtex, skip this section — that side is not this script's business. This is for when the reference list has to live in the manuscript.

```
:::{thebibliography widest="99"}

\bibitem{ndh2024} Nova de Hi, 2024, *What is markdown?*

\bibitem{kitty2026} Kitty the Cat, *Please Make me Pleasant,* 2026

:::

\begin{thebibliography}{99}
\bibitem{ndh2024} Nova de Hi, 2024, \emph{What is markdown?}

\bibitem{kitty2026} Kitty the Cat, \emph{Please Make me Pleasant,} 2026
\end{thebibliography}
```

`widest` is the argument of `\begin{thebibliography}{...}` — the **widest label**, which sizes the indent. Match its digit count to the number of entries: `9` for nine or fewer, `99` below a hundred. It defaults to `99`.

The entries themselves are written as plain `\bibitem`. There is deliberately no markdown syntax for them, for two reasons. First, inline formatting such as `*italics*` already works in the text after `\bibitem`, so the only thing you type by hand is the `\bibitem{key}` fragment. Second, using the list's numbering as citekeys means reordering the list once silently repoints every citation at the wrong work. Let LaTeX assign the numbers and keep meaningful names as keys.

The `\bibitem[display label]{key}` form (natbib author-year labels) passes through as well.

17.3. Verse — `hverse` and `verse`

The contents of an ordinary environment are processed like body text, which means consecutive lines not separated by a blank line are **joined into one paragraph**. That is right for prose and wrong for poetry, where the line break *is* the meaning.

`hverse` and `verse` are therefore special-cased: the **line structure of the source is passed through as written**.

```
:::{hverse}

Roses are red,
Violets are blue,

Sugar is sweet,
And so are you.

:::
```

```
\begin{hverse}
Roses are red,
Violets are blue,

Sugar is sweet,
And so are you.
\end{hverse}
```

- Blank lines survive as **stanza breaks**.
- The blank line right after `:::` and right before the closing `:::` is trimmed. Leaving that space is nicer to read in the manuscript, but passed through it would produce an empty stanza.
- Inline formatting and macros still work — `**bold**`, Josa macros, footnotes. Only the joining of lines is suppressed; each line is processed as usual.
- Block syntax (lists, tables, nested `:::`) does not work inside, because the block parser is bypassed entirely.

`hverse` comes from oblivoir-misc and needs no `\\` at the end of a line. Standard `verse` does, so it is appended automatically — except on the last line of a stanza.

```
\begin{verse}
Roses are red, \\
Violets are blue,

Sugar is sweet, \\
And so are you.
\end{verse}
```

Any other environment can be given the same treatment from `mapping.yaml` with `lines: keep`, plus `line_end` if something must terminate each line.

```yaml
environments:
  mypoem:
    begin: '\begin{mypoem}'
    end: '\end{mypoem}'
    lines: keep
    line_end: ' \\'
```

17.4. Completely raw — `lines: raw`

Even `lines: keep` still runs inline formatting and macros on every line. For some environments that processing itself is the problem — in particular, environments that embed raw LaTeX or code in another language verbatim.

TikZ is the standard example.

```
:::{tikzpicture raw="remember picture,overlay"}
\draw[very thick,->,>=latex] (NoteY-S)to[out=-90,in=90](NoteZ-N) ;
:::
```

The fragment `[out=-90,in=90](NoteZ-N)` happens to have the exact shape of Markdown's `[label](url)` link syntax, so with no special handling it gets rewritten into `\href{NoteZ-N}{out=-90,in=90}`. Drawing code can contain `%`, `_`, `$`, `**` — anything Markdown touches — so this collision is not unique to TikZ.

`lines: raw` skips inline processing entirely and pastes the source through verbatim, indentation and line breaks included.

```
environments:
  tikzpicture:
    begin: '\begin{tikzpicture}[%(raw)s]'
    end: '\end{tikzpicture}'
    lines: raw

\begin{tikzpicture}[remember picture,overlay]
\draw[very thick,->,>=latex] (NoteY-S)to[out=-90,in=90](NoteZ-N) ;
\end{tikzpicture}
```

- Macros, math, footnotes, index entries, inline formatting — **none of it is interpreted.** By definition everything here is raw.
- The protection applies only **inside** that environment. Once the document continues past it, everything else is processed as usual.
- This is the opposite of `lines: keep`. Use `keep` when each line should still allow things like `**bold**` or Josa macros (verse); use `raw` when nothing at all should be touched (drawing code, embedded languages).
- This is unrelated to the `%(raw)s` placeholder (section 7): that one passes an **attribute value** on the opening line through raw, while `lines: raw` treats the environment's **entire body** as raw. The two can be combined, as in the example above — the `raw="..."` attribute supplies the bracket contents of `\begin{tikzpicture}[...]`, and `lines: raw` protects the drawing code inside it.

17.5. Optional attributes — defaults and ? lines

`%(key)s` makes an attribute **mandatory**. If it is missing, the whole template falls back to a bare `\begin{name}` with a warning. That is not enough for attributes like a caption or a label, which may or may not be there, so templates accept two further notations.

Notation	Meaning
`%(key|default)s`	Substitute *default* when the attribute is absent
A leading `?` on a line	**Drop the whole line** if any attribute it references is empty

The `?` is stripped when the line is kept. For a line that really does start with a question mark, write `??`. The fallback behaviour for an attribute with neither a default nor a `?` is unchanged.

A default applies when the attribute is **absent**, not when it is empty. So an explicit `align=""` suppresses the default, and if that placeholder sits on a `?` line the line disappears altogether.

Both notations work in the `end` template too. That is how `figure` in 17.1 places its caption below the content.

The built-in `table` above is defined with exactly these notations:

```
# the built-in definition of the table environment
environments:
```

```
table:
  begin: |-
    \begin{table}[%(place|htbp)s]
    ?%(align|\centering)s
    ?\caption{%(caption)s}
    ?\label{%(label)s}
  end: '\end{table}'
```

Omit an attribute and only its line disappears.

- `:::{table caption="Just a caption"}` — drops the `\label` line and uses the `htbp` default for `place`.
- `:::{table}` — with no attributes at all, only `\begin{table}[htbp]` and `\centering` remain.

Beware that a value like `align="center"` is not a LaTeX command and will emit the literal word `center` — the script does not interpret attribute values.

---

18. Characters that are auto-escaped

Any of the following characters, written plainly in body text (i.e. not caught by any of the protection rules above), is automatically converted to a safe LaTeX representation.

Character	Output
`\`	`\textbackslash{}`
`&`	`\&`
`%`	`\%`
`#`	`\#`
`_`	`\_`
`$`	`\$`
`{`	`\{`
`}`	`\}`
`^`	`\textasciicircum{}`

In other words, unless you intend it as math or a macro, it's safe to just type `$`, `%`, `&`, `_`, etc. directly into markdown body text.

`~` (tilde) is deliberately absent from this table. Neither `~` nor `\~` is escaped; both pass through to LaTeX as written — see 7.2 for what that means and why.

18.1. `^^M` — forcing a line break in the source itself

Writing `^^M` inserts an actual newline character **into the generated .tex file itself**, right at that spot.

```
\linecommand argument goes here^^Mthe next sentence continues normally.
```

```
\linecommand argument goes here
the next sentence continues normally.
```

- This dialect collapses every line break in body text into a space (a paragraph always comes out as one line), so there was previously no way to use a TeX macro like `\def\linecommand#1^^M{...}` that takes **everything up to the end of the input line** as its argument. `^^M` is exactly that escape hatch, meant for this rare case. The notation itself is borrowed directly from TeX's own caret notation for the newline character (carriage return, character code 13).

- **This is purely about a newline in the source — it is not a request for LaTeX to break the line visually.** For a visible line break, use `\\` or `\newline` (section 6) instead. `^^M` only makes TeX see that spot as the end of a line in the file.
- **It looks similar to `^^emphasis^^` (the dotted-emphasis mark from the default `extra_inline`) but doesn't conflict with it.** That pattern only matches when there's a closing `^^`, and `^^M` is an unpaired pair of carets plus an M, so it never matches that rule. This substitution also finishes before that one runs, so even if a real `^^emphasis^^` appears later in the same text, its carets can't get mismatched with `^^M`'s.
- Inside an inline code span (`` `^^M` ``) or a code fence, it stays literal text — safe to use freely in examples.

---

19. Overriding the mapping with `mapping.yaml`

This is deep-merged over the built-in `DEFAULT_CONFIG`, so you only need to specify the keys you want to change.

```yaml
# mapping.yaml
headings:
  1: part
  2: chapter
inline:
  - name: bold
    pattern: '\*\*(.+?)\*\*'
    template: '\textbf{%s}'
environments:
  warning:
    begin: '\begin{tcolorbox}[colback=red!5,title=Warning]'
    end: '\end{tcolorbox}'
refcite:
  "name:": '\hbref{%s}'
codeblock_template: "\\begin{lstlisting}[language=%(lang)s]\n%(body)s\n\\end{lstlisting}"
hr: '\hrulefill'
showexam:
  ratio: 0.42
  style: lstlisting
  frame: none
```

Set it via `mapping: mapping.yaml` in the front matter, or pass `--config mapping.yaml` on the CLI (`--config` takes priority).

Dictionary-shaped keys merge in per-entry, for free. `environments` and `refcite` are like this — they are `{name: value, ...}` dictionaries, so the deep merge combines them entry by entry. That's what happens with `environments.warning` above: only `warning` was written, yet `quote`, `table`, and the other built-in environments stay exactly as they were. The "extra_" keys (`extra_macros`, `extra_inline`) exist for the opposite reason — `literal_macros` and `inline` are **lists**, which get replaced wholesale, so a separate append-only key was needed to add an entry without copying the whole default over. `environments` is already a dictionary and needs no such key (no `extra_environments`).

19.1. Right in the manuscript — `mappingrule:`

When only one or two entries need changing, a separate file is more than the job calls for. `mappingrule:` takes a YAML fragment written directly in the front matter and merges it in place.

```
---
title: Example
class: article
mappingrule: |
  environments:
    warning:
      begin: '\begin{tcolorbox}[colback=red!5,title=Warning]'
      end: '\end{tcolorbox}'
  extra_inline:
    - name: doubleline
      pattern: '\=\=(.+?)\=\='
      template: '\uuline{%s}'
  ellipsis: '\kswdashCdots{}'
  hr: '\fancybreak{* * *}'
---
```

- It follows the same rules as any other front matter key: it is inherited through `setup:`, and `mappingrule+:` appends to whatever was inherited (section 21).
- Precedence layers **on top of** a `mapping:` file, with `--config` layered on top of that in turn — `--config` > `mappingrule:` > `mapping:`. This mirrors the reasoning behind `--config` beating `mapping:`: a command-line flag states its intent most plainly.
- Broken YAML, or anything that isn't `key: value`, stops the run with an error — a mapping setting that silently gets ignored is hard to trace back.

The point is that copying an entire `mapping.yaml` over is unnecessary. Write only the entries that change; every other default stays as it was.

---

20. File inclusion (`{{filename}}`)

Modeled after multimarkdown's transclusion syntax: it splices the body of another `.md` file in place. Useful for assembling a book out of separate chapter files under one top-level document.

```
# Preface

Preface body text.

{{chapter1.md}}

{{sub/chapter2.md}}

{{/absolute/path/appendix.md}}
```

- **The included file's YAML front matter is ignored.** Only its body is spliced in — anything the included file sets (`title`, `mapping`, etc.) has no effect on the top-level document's settings.
- **The extension must be written out exactly.** Omitting it, as in `{{chapter1}}`, does *not* trigger any automatic `.md` fallback — even if `chapter1.md` exists on disk, the tool looks for a file literally named `chapter1` and errors out if that's not found.
- **Path rules**: a bare filename with no path, e.g. `{{chapter1.md}}`, is resolved relative to **the directory of the including file (i.e. the top-level input file)**. A relative path with directories, e.g. `{{sub/chapter2.md}}`, is likewise resolved against the top-level input file's directory. An absolute path is used as-is.

- If the file can't be found, conversion aborts: it prints an error of the form "포함 파일을 찾을 수 없습니다: `{{...}}` (확인한 경로: ...)" ("could not find the include file... (path checked: ...)") to stderr and exits with status 1.
- **Inclusion is resolved recursively — if an included file itself contains `{{a-sub-sub-file.md}}` (and that one contains another, and so on), it keeps digging in.** Relative paths are always resolved against the **top-level input file's** directory, regardless of nesting depth — a file nested two or three levels deep still has to spell out its relative paths as if from the top-level file, not from its own folder ("no matter how many levels deep the inclusion goes, paths are always measured from one fixed point" is the consistent stance here).
  - **A circular inclusion** (A includes B and B includes A again — including a file including itself directly) halts immediately with an error that shows the whole chain of what included what.
  - **Very deep recursion should be avoided.** Even without a cycle, a chain longer than 20 levels halts with an error — this is purely a safety net; no normal manuscript needs to go this deep. Because the path reference point is always the single top-level file, going deeper makes it progressively harder for the author to trace "where did this content actually come from" — keeping nesting to 2–3 levels is recommended.
  - `--lint` and `--syncinfo` follow this recursion too — every sub-file gets linted no matter how many levels the split goes, and PDF ↔ md reverse search (section 1.5) guides accurately down to a sub-sub-file. `ndhmain:` (section 16) works the same way regardless of nesting depth too — it only needs to know which top-level document a sub-file ultimately belongs to, so it doesn't matter how many intermediate levels sit in between.
- A `{{...}}` inside a code fence (```) or inline code immediately adjacent to backticks (`` `{{file.md}}` ``) is treated as a syntax example and is not substituted — same convention as the raw-protection rules in sections 9 and 10.
- **Write `{{file.md}}` as its own line, with blank lines before and after it.** The included file may contain multiple paragraphs or headings, and if the directive is embedded mid-paragraph with other text, block parsing may not behave as expected.

20.1. Splicing in a raw LaTeX file verbatim (`{{tex:filename}}`)

Prefixing the name with `tex:`, as in `{{tex:filename.tex}}`, splices that file into the output **without changing a single character** of it and without treating it as markdown at all. Use this for a hand-crafted title page or any other LaTeX fragment you want dropped into the document exactly as written.

```
{{tex:titlepage.tex}}

# Introduction

body...
```

- Markdown processing is skipped **entirely**. `*`, `_`, `[text](url)`, `{{another include}}` — any of this dialect's syntax inside that file is left as literal text; none of it is turned into emphasis, links, or a nested inclusion.
- **There's no such thing as YAML front matter here.** Unlike a plain `{{file.md}}`, no attempt is made to peel off a leading header — even if the `.tex` file happens to start with `---`, that line is spliced in exactly as written.
- **Things like `\documentclass`, `\begin{document}`, or `\end{document}` inside the file are left completely untouched** — the script doesn't strip or validate any of that. What goes into that file is entirely the author's responsibility, which is this dialect's guiding principle here; if the result fails to compile, that's a problem with the file's contents, not something the script should be fixing on your behalf.

- Path rules and error handling match the plain `{{file.md}}` form in section 20 (relative to the top-level input file, absolute paths supported, aborts with an error if not found).
- Recursive inclusion isn't supported here either — any `{{...}}` inside a `tex:`-included file stays literal (unsurprising, since the file was never read as markdown in the first place).
- `--syncinfo` approximates the whole thing as "that tex file itself, line 1" — since the source was never markdown to begin with, "which exact line" has no meaningful answer. A reverse search lands you in that tex file.
- Also works in `--pandoc` mode (it's wrapped in pandoc's raw-attribute code fence, so pandoc doesn't interpret it as markdown either).

20.2. Several files at once (`{{file-pattern;question-marks=list}}`)

Files numbered `sub01.md` through `sub20.md` don't have to be spelled out on twenty separate lines — one line does it.

```
{{sub??.md;??=01-20}}
```

This is exactly equivalent to writing `{{sub01.md}}` through `{{sub20.md}}`, each separated by a blank line — since transclusion is recognized most reliably as "its own line with blank lines before and after," that's precisely how it gets expanded.

Listing several comma-separated pieces allows discontinuous ranges too.

```
{{sub??.md;??=01-10,12-20}}
```

`01` through `10`, then `12` through `20` — `11` is left out exactly as written. A piece can be a single number (`7`) or a `start-end` range (`01-10`), and the two can be mixed: `?=1-3,7,9-10` works too. With only one piece (no comma), this looks exactly like the plain range syntax, so anything written as `{{sub??.md;??=01-20}}` before comma support existed keeps working unchanged. Pieces are concatenated in the order they're written — nothing is sorted or de-duplicated, so deliberately overlapping ranges (`1-5,3-7`) will produce 3, 4, and 5 twice.

- **The number of question marks decides the padding width.** `??` always pads to 2 digits (01, 02, …, 20). A single `?` means no padding at all (1, 2, …, 9). Numbers that outgrow the width (as in `{{sub?.md;?=1-15}}`) aren't truncated — they come out naturally as 1, 2, …, 9, 10, …, 15. Padding only ever guarantees a *minimum* width; it never shortens a number that's already wider. This one width applies uniformly across the whole list.
- **The `?`/`??` positions are numeric only.** This is not shell-glob `?` (matching "any single character") or a regex wildcard — it's strictly a syntax for zero-padding an integer (or range) to a fixed width, not general pattern matching. Character ranges (`a-z`) or any other kind of wildcard are not supported.
- It's an error if the question marks don't appear in the filename pattern (as in `{{sub##.md;??=01-20}}`), if any piece of the list isn't a valid number or range, or if a range's start is greater than its end.
- Works together with the `tex:` prefix too, as in `{{tex:sub??.tex;??=01-03}}` — each expanded name (`tex:sub01.tex`, etc.) follows section 20.1's rules exactly as written.
- `--lint` checks the existence of each expanded name individually, and the recursive sub-file check (see the `--lint` discussion near section

  1. picks up every expanded file too.
- `--syncinfo` is fully supported — each expanded file's content is tracked back to its own correct source line numbers. Only the blank line inserted between files has no real answer to give, so it's approximated (meaning "roughly around here").
- This syntax isn't recursive expansion in itself — each expanded name is read exactly the way an ordinary single `{{file}}` would be (every other rule in section 20 still applies).

When you don't know where it ends, use `*` (an open-ended range).

```
{{contents/sub??.md;??=01-*}}
```

Starting from `sub01.md`, this automatically goes up to whatever the highest number actually present in that folder is (up to `sub27.md`, say, if that's the last one there). No need to hand-edit the end number every time the file count changes.

- **Gaps in the middle are skipped silently.** If `sub11.md` is missing, `sub10.md` is simply followed by `sub12.md` — because this actually scans the directory and collects whatever files exist.
- **This is a different philosophy from a fixed range (`01-20`).** A fixed range still stops with a loud error if any file inside it is missing (so typos don't get silently waved through) — writing `*` is itself an explicit signal of "give me whatever's actually there," so only this case gets to skip quietly.
- `*` is only accepted **at the end**. Putting it at the start (`*-20`) or on both sides (`*-*`) is ambiguous and isn't supported.
- The search is scoped to **the directory relative to the current document** — it doesn't reach into `setup:`-inherited locations or shared resource folders the way ordinary transclusion search does. Writing `contents/sub??.md` searches only inside that subfolder.
- The number of question marks still decides "how many digits count" — with `??`, a 3-digit `sub100.md` is excluded from the search.
- If nothing matching is found at all (a mistyped folder name, say), that's still an error — an open-ended range only skips gaps, it doesn't silently accept finding nothing.
- Works as one piece of a comma-separated list too (`?=1-3,7-*`), and together with the `tex:` prefix.

---

21. Inheriting a master header (`setup:`)

Front matter for a LaTeX document gets long fast, and once you produce several documents in the same format those twenty-odd lines end up duplicated in every file. The `setup:` key lets you factor that shared part into a separate file and inherit it.

```
---
setup: mymaster.md
title: A title just for this document
---
```

This is **exactly symmetric** with the `{{filename}}` inclusion of section 20.

Syntax	Takes	Discards
`{{file.md}}`	that file's **body**	its front matter
`setup: file.md`	that file's **front matter**	its body

A setup file may therefore take either of two forms.

- **A complete markdown document**: if it has front matter wrapped in `---`, only that is read and the body is discarded wholesale. In other words, **you can point at an already-typeset document and use it as a style donor** — "same format as that last book" becomes a one-liner.
- **A header-only file**: with no `---` fences, the entire file is parsed as YAML. The extension is irrelevant (`.yaml`, `.txt`, whatever).

21.1. Merge rule — local wins

When the same key appears on both sides, **the current file's value replaces the setup file's.**

```yaml
# base.yaml
class: oblivoir
classoptions: 11pt, a4paper
author: Nova de Hi
preamble: |
  \usepackage{kotex}
---
setup: base.yaml
class: memoir          ← replaces oblivoir
---
```

21.2. To append instead, use `key+:`

To **add to** an inherited value rather than replace it, suffix the key name with `+`. This is the form you'll reach for most often — keeping the shared preamble intact while piling one more package onto this document.

```yaml
---
setup: base.yaml
preamble+: |
  \usepackage{tikz}
classoptions+: twoside
---
```

```
\documentclass[11pt, a4paper, twoside]{oblivoir}
\usepackage{kotex}
\usepackage{tikz}
```

- The setup file's value comes first, the current file's is appended after it. Thanks to that ordering, overriding an individual definition is handled by LaTeX's own `\renewcommand` — there's no need to reimplement override semantics at the YAML layer.
- The separator is a comma plus space for `classoption`/`classoptions`, and a newline for everything else.
- If a single file has **both** `preamble:` and `preamble+:`, the replacement is applied first and the append lands on top of the result.
- Using `key+:` in a document with no `setup:` at all simply behaves like an ordinary `key:`.
- Loading the same package twice in the preamble is generally harmless, since LaTeX's `\usepackage` already deduplicates.

21.3. Listing several files

`setup:` accepts not just a single file but also a **list**. Useful when the page layout comes from a shared header and something document-specific (a glossary's index settings, a slide deck's margins) needs to sit on top of it.

```yaml
---
setup: [base.yaml, glossary.yaml]
title: This document's own title
---
```

- They merge left to right. **The file listed later overrides the one before it** — the same override rule that applies to front matter in general (21.1).
- `key+:` keeps appending across the whole list. If `glossary.yaml` has a `preamble+:`, it lands after `base.yaml`'s `preamble:`, and if the top-level document itself then has its own `preamble+:`, that appends on top of that in turn.
- Any file in the list can have **its own `setup:`** (the chain rule from 21.6 applies just the same). In that case, its own ancestors are layered onto whatever has accumulated so far, and only then is its own content applied — sibling relationships within the list are treated as part of the inheritance chain.
- **Listing the same file twice is treated as a circular reference.** `setup: [a.md, a.md]` means nothing and is almost always a typo.
- On the CLI, list several files with **commas** — `--setup base.yaml,glossary.yaml` (not spaces, so it can't be confused with the input-file argument). A CLI `--setup` still replaces the document's own `setup:` wholesale, exactly as with a single file.

21.4. `title` is not inherited

`title` is a document's **identity**, not its style, so it is never inherited from a setup file. If you point at a finished document as a style donor and its title silently tagged along, the result wouldn't be an error — it would be *a PDF with the wrong title on it*. Every other key (`author`, `date`, `class`, ...) is inherited normally.

21.5. `mapping:` is always resolved against the top-level file

There is **always exactly one** effective mapping configuration; multiple mapping files are never merged. The precedence is:

```
--config (CLI)  >  the current file's mapping:  >  a setup file's mapping:
```

Even when the value is inherited from a setup file, **a relative path is always resolved against the top-level input file's directory** — not against the directory holding the setup file. If a shared setup file needs to point at a mapping file sitting next to it, use an absolute path. For the same reason `mapping+:` is not supported; using it prints a warning and is ignored.

21.6. Chains are capped at two levels

A setup file may carry its own `setup:` key. That path is resolved against the directory of **the file that declares it** (unlike `mapping:`).

```
doc.md      →  series.md        →  base.yaml
(your file)    (series-wide)       (house default)
```

That three-tier arrangement is the deepest allowed; anything beyond it aborts with an error. The cap is deliberate — past that point you can no longer trace where a header value was inherited from. Circular references (`a.md` → `b.md` → `a.md`) are caught as errors too.

The reason a setup file's own `setup:` is allowed at all, rather than being quietly ignored, is that a finished document used as a style donor is quite often built on a shared header itself. Ignoring it would pull in **only half the style, with no error at all** — silently wrong output.

21.7. Tracing where a value came from

Because values can arrive from several places, use `--dump-meta` to inspect the merged result.

```
$ python3 ndhmd4tex.py doc.md --dump-meta
# setup 체인: doc.md -> shared/series.md -> shared/base.yaml
author: Nova de Hi
class: oblivoir
classoptions: 11pt, a4paper, twoside
preamble: '\usepackage{kotex}

  \usepackage{tabularray}'
title: My Document
```

(The tool's own diagnostics are in Korean, so the first line reads `# setup 체인:` — "setup chain:" — followed by the files it actually read, in order.)

If a setup file can't be found, the chain is too deep, or there's a circular reference, an error is printed to stderr and the tool exits with status 1 — the same convention as the inclusion errors in section 20.

Also note that the front matter of a `{{}}`-included file is discarded wholesale, so putting `setup:` in an included file has no effect whatsoever.

---

22. Known limitations / caveats

- The older description-list form (`term :: definition`) cannot be told apart from prose that genuinely contains that shape, as in `he said :: no`. Use the braced form (`{term} :: definition`) or shut the older one off with `description_syntax: braces` (see section 11).
- The `:::{name}` custom-environment syntax **is not recognized if there's a space** between `:::` and `{`.
- To put a separate block — a list, heading, etc. — inside a blockquote (`>`), you need a bare `>` line to mark the blank-line separator. Otherwise everything just gets merged into one paragraph.
- When wrapping raw LaTeX syntax in inline-code backticks purely to show it as an example — e.g. `` `\begin{...}...\end{...}` `` — the backticks need to be **immediately adjacent** to `\begin`/`\end` to be handled safely. If there's padding space between the backtick and the content (as in `` ` \begin{...}...\end{...} ` ``), it can still be misidentified as a real raw environment.
- For the same reason, showing `<!-- -->` itself as an example via inline code only stays safe if the backticks sit immediately next to it (padding space can get it misidentified as a real comment and stripped).
- Tables use a fixed width based on the header row's column count.
- Inside blocks that are protected as raw (code fences, display math, raw environments), none of this dialect's syntax — bold, quote conversion, etc. — is ever applied, since that content is assumed to already be finished LaTeX.
- A list item **cannot contain block elements** such as paragraphs or code blocks. A second paragraph separated by a blank line closes the list and escapes to top level (see section 11). A nested list is the only block that can live inside an item.
- `{{filename}}` file inclusion is **not resolved inside code fences or inline code spans** — it stays literal there. So showing another template syntax such as `` `{{ user.name }}` ``, or putting `{{` inside a raw span from 7.1, is safe.
- `{{filename}}` file inclusion supports **only one level**. If an included file itself contains `{{...}}`, it is not resolved recursively and is left as literal text (see section 20).

- `setup:` chains are capped at **two levels** (i.e. `your file → setup → setup` at most). Going deeper is an error (see 21.6).
- A `mapping:` value inherited via `setup:` has its relative path resolved against **the top-level input file's** directory, not the setup file's. When a shared setup file needs to point at a mapping file, an absolute path is safer (see 21.5).
- `title` is not inherited through `setup:` (see 21.4). `author` and `date`, by contrast, *are* — so if you use a finished document as a style donor and the author differs, override it explicitly in the current file.
- `showexam` boxes **cannot break across a page**. A long example is pushed wholesale onto the next page, and switching to `tcolorbox` does not help because `breakable` is incompatible with `sidebyside` (see 10.2).
- A `showexam` block indented inside a list item or blockquote is not supported — use it at the top level only.
- `showexam` with `style: verbatim` cannot wrap long lines, so they overflow the box. Use `minted` or `lstlisting` if you need wrapping. If the code mixes in Unicode (Korean, etc.), `lstlisting` cannot handle it and `minted` is effectively the only option (see section 10).

---

23. Beamer frames (`={4,}{attributes}`)

For `class: beamer` presentations, this saves wrapping every frame by hand in `\begin{frame}`/`\end{frame}`. It's exactly the same convention as `:::{environment}` (section 10) — braces only on the opening side, a bare marker on the closing side — applied to equals signs.

```
======{title="Frame title here",subtitle="A smaller subtitle",opt=fragile}
Body content.
======
```

```
\begin{frame}[fragile]
\frametitle{Frame title here}
\framesubtitle{A smaller subtitle}

Body content.
\end{frame}
```

- **The opening line is 4+ equals signs plus a brace attribute block** (`={4,}\{...\}`); **the closing line is 4+ equals signs alone** (no braces). Requiring at least 4 mirrors the same logic behind reserving `---`/`***` for horizontal rules.
- The attribute syntax is exactly `:::{environment}`'s — `key="value"`/`key=value`, comma-separated — no new parsing rules were invented for this.
- **`name=` swaps out `frame` for a different environment.** The value goes through unprocessed (it's a LaTeX command-name slot, not on-screen text), and gets used as-is on both `\begin{...}` and `\end{...}`. This is for Beamer extension packages like `ltx-talk`, whose `frame*` or `itemizedframe` environments behave like `frame` but under a different name — title/subtitle/opt/raw handling stays exactly the same regardless of the environment's name.

```
====={name="frame*",title="TITLE"}
Body content.
====
```

```
  \begin{frame*}
  \frametitle{TITLE}

  Body content.
  \end{frame*}
```

- `title=`/`subtitle=` become `\frametitle{}`/`\framesubtitle{}` respectively (the value goes through inline processing), and `opt=` goes straight into `\begin{environment}[...]`'s brackets unprocessed. `raw=` is appended directly after `\begin{environment}[opt]` on the same line, completely unprocessed (with no `opt`, it's appended right after `\begin{environment}` with no brackets at all) — the same reserved-word treatment as `:::{environment}`'s own `raw` attribute. With none of the four given, the corresponding line or piece simply doesn't appear.
- **If more than one bracket or brace group is needed, tack the rest onto `raw=`.** `opt=` only ever fills in the **first** `[...]`. Beamer sometimes wants two brackets, as in `\begin{frame}[<+->][fragile]` (a default overlay specification plus ordinary options) — rather than inventing another attribute for that, just put the second bracket straight into `raw=` as a literal string; it gets appended right where it's written. See below for an example.

```
======{opt=<+->,raw="[fragile]",title="Itemize"}

\begin{frame}[<+->][fragile]
\frametitle{Itemize}
```

Verified (xelatex + beamer) that `[<+->]` applies as the frame's default overlay behavior, revealing list items one at a time with no `\pause` needed. Whatever combination of brackets or braces is needed, and however many, they can all be strung together in one `raw=` string in the order they should appear — `raw="[fragile]{Extra title}"`, for instance.

- **The inside of a frame is entirely ordinary body content.** Lists, description lists, tables, blockquotes, nested `:::{environment}` blocks — every block-level construct this dialect supports works exactly as it does outside a frame, because (just like `:::{environment}`) the inner content is handed straight back into recursive block processing rather than being run through a "frame-specific parser." That's also why a blank line isn't needed right after the opening line even when the source doesn't have one — the recursive call already treats that first line as the start of a fresh block.
- **Combined with description lists, this gets you Beamer's overlay specification syntax.** Since description lists can carry a `{!option}` block on their first item's definition (section 11), you can write `\begin{description}[<+-| alert@+>]` to have items get revealed and highlighted one step at a time. See below for an example.

```
{uncover} ::  {!"<+-| alert@+>"} visible on the given slides.
{only} ::  show on the given slides.

\begin{description}[<+-| alert@+>]
\item[uncover] visible on the given slides.
\item[only] show on the given slides.
\end{description}
```

Verified by actually compiling with xelatex + beamer: each overlay step highlights exactly the right item with `\alert`, one at a time.

- `--lint` catches unbalanced frames (never closed, or a close with no matching open) and whitespace between `====` and `{` (which stops it from being recognized as a frame at all).
- `--syncinfo` tracks an entire frame as a single block — reverse search lands you exactly at the frame's own starting position in the source, but doesn't pinpoint a specific line inside it (the same limitation `:::{environment}` has).

- **There's no separate default configuration for `class: beamer`.** This dialect's default `classoptions` (e.g. `chapter`) are meant for book/article-style classes and may include options beamer doesn't recognize — set `classoptions: ""` explicitly in front matter when typesetting with beamer.